

Chapter 7 Multimedia ~~Networking~~

A note on the

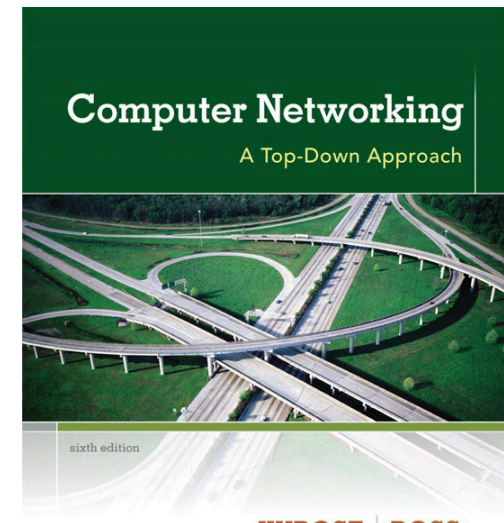
We're making these

They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a lot of work on our part. In return for use, we only ask the following:

- ❖ If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- ❖ If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

©All material copyright 1996-2012
J.F Kurose and K.W. Ross, All Rights Reserved



*Networking: A
Top Down
Approach
6th edition
Jim Kurose, Keith
Ross
Addison-Wesley
March 2012*

Multimedia networking: ~~outline~~

7.1 multimedia networking
applications

7.2 streamin o

7.4 pr
conversational applications

7.5 network support for multimedia

Multimedia networking: ~~outline~~

7.1 multimedia networking
applications

7.2 streamin 0

7.4 pr
conversational applications

7.5 network support for multimedia

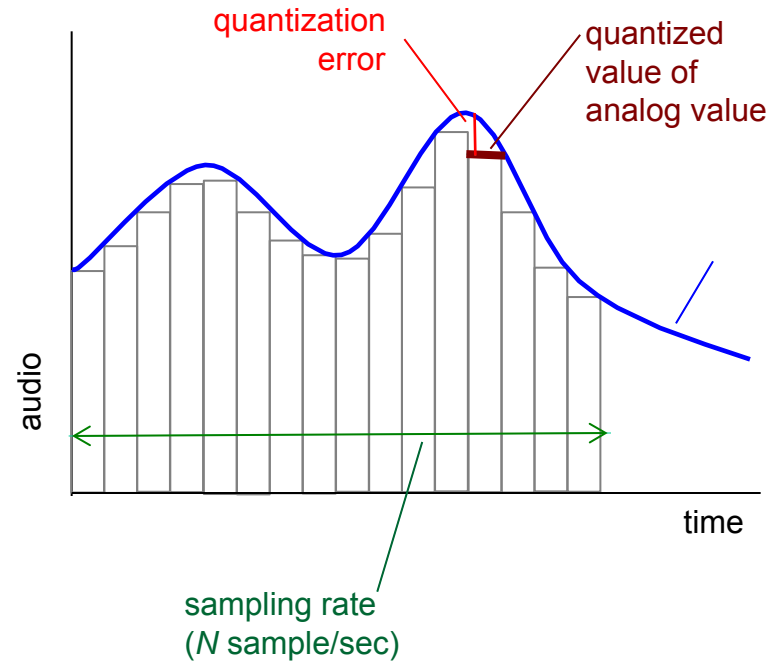
Multimedia: ~~audio~~

- ❖ analog audio signal sampled at constant rate

- telephone: 8

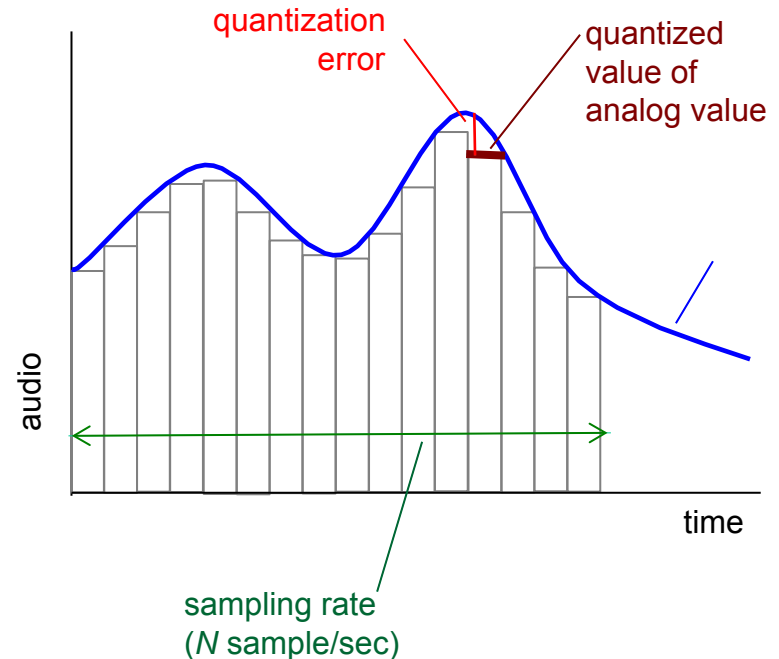
sa

- ❖ each sample quantized, i.e., rounded
 - e.g., $2^8=256$ possible quantized values
 - each quantized value represented by bits, e.g., 8



Multimedia: ~~audio~~

- ❖ example: 8,000 samples/sec, 256 quantized values: 64,000 bps
 - so reduction
- ❖ example rates
- ❖ CD: 1.411 Mbps
- ❖ MP3: 96, 128, 160 kbps
- ❖ Internet telephony: 5.3 kbps and up



Multimedia: ~~video~~

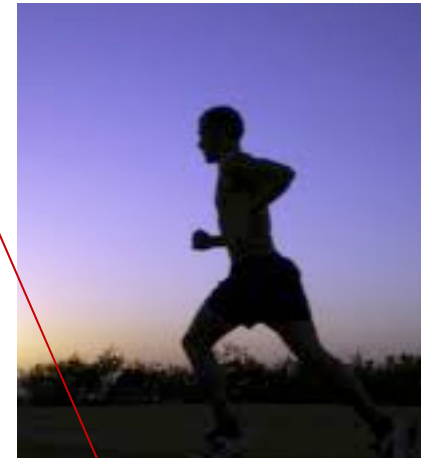
- ❖ video: sequence of images displayed at constant rate
 - e.g. 24 image
- ❖
 - eac represented by bits
- ❖ coding: use redundancy *within* and *between* images to decrease # bits used to encode image
 - spatial (within image)
 - temporal (from one

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i



frame $i+1$

Multimedia: ~~video~~

- ❖ *CBR: (constant bit rate):* video encoding rate fixed
- ❖ *VBR: (variable bit rate):* video enco

tempor
changes

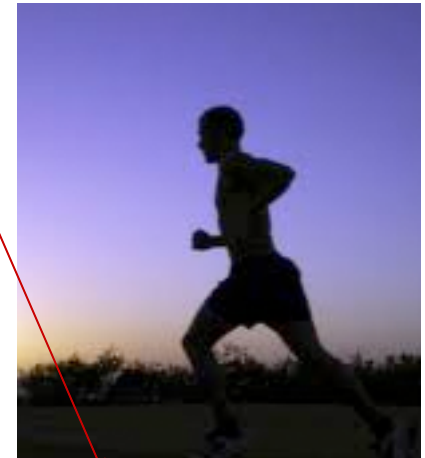
- ❖ *examples:*
 - MPEG 1 (CD-ROM) 1.5 Mbps
 - MPEG2 (DVD) 3-6 Mbps
 - MPEG4 (often used in Internet, < 1 Mbps)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i



frame $i+1$

Multimedia networking: 3 application types

- ❖ *streaming, stored* audio, video
 - *streaming*: can begin playout before downloading entire file
 - *stored (at server)* can be accessed and played back much more quickly than streaming, but not as fast as stored locally
 - e.g. YouTube
- ❖ *conversational* voice/video over IP
 - interactive nature of human-to-human conversation limits delay tolerance
 - e.g., Skype
- ❖ *streaming live* audio, video
 - e.g., live sporting event (futbol)

Multimedia networking: ~~outline~~

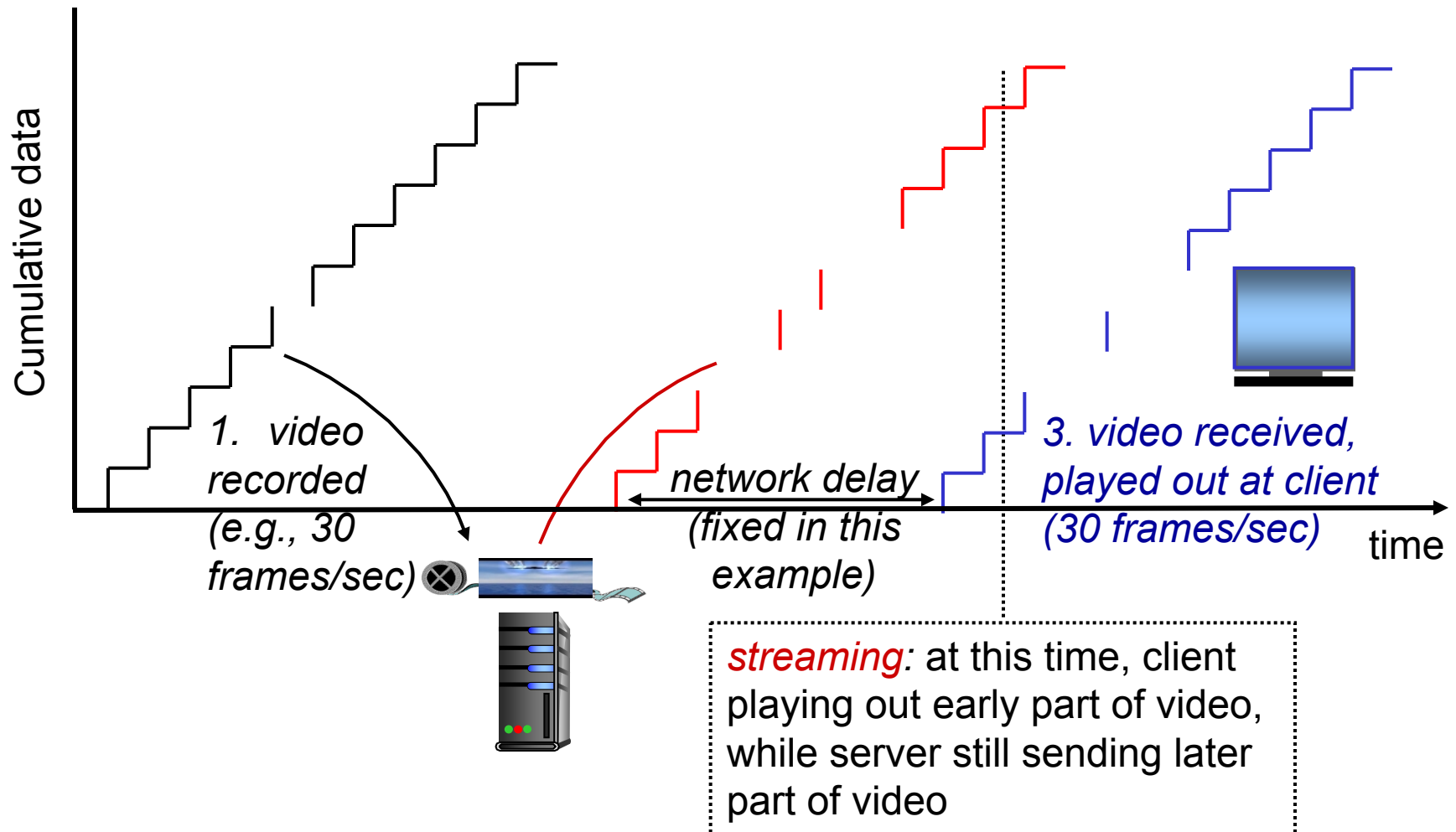
7.1 multimedia networking
applications

7.2 streamin o

7.4 pr
conversational applications

7.5 network support for multimedia

Streaming stored video:



Streaming stored video: ~~challenges~~

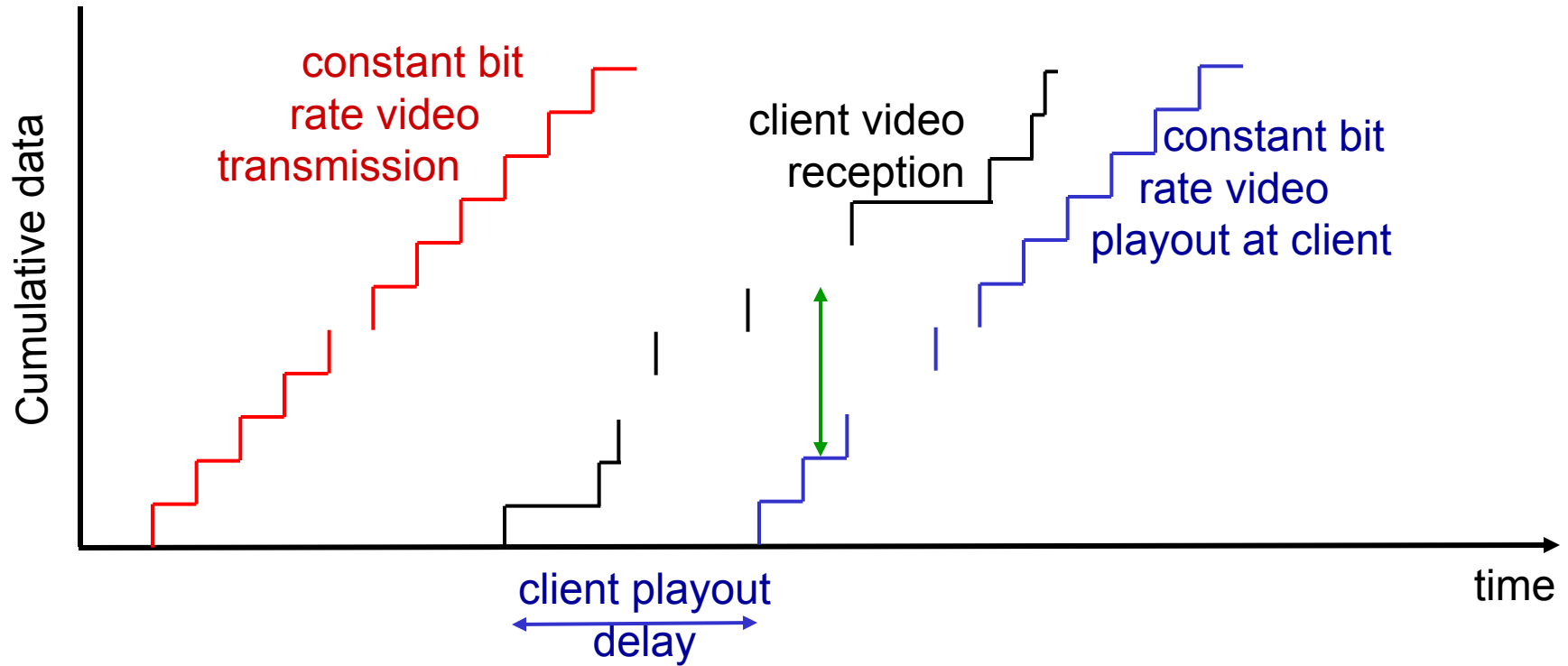
- ❖ *continuous playout constraint*: once client playout begins, playback must match original

(jitter)

buffer to match playout requirements

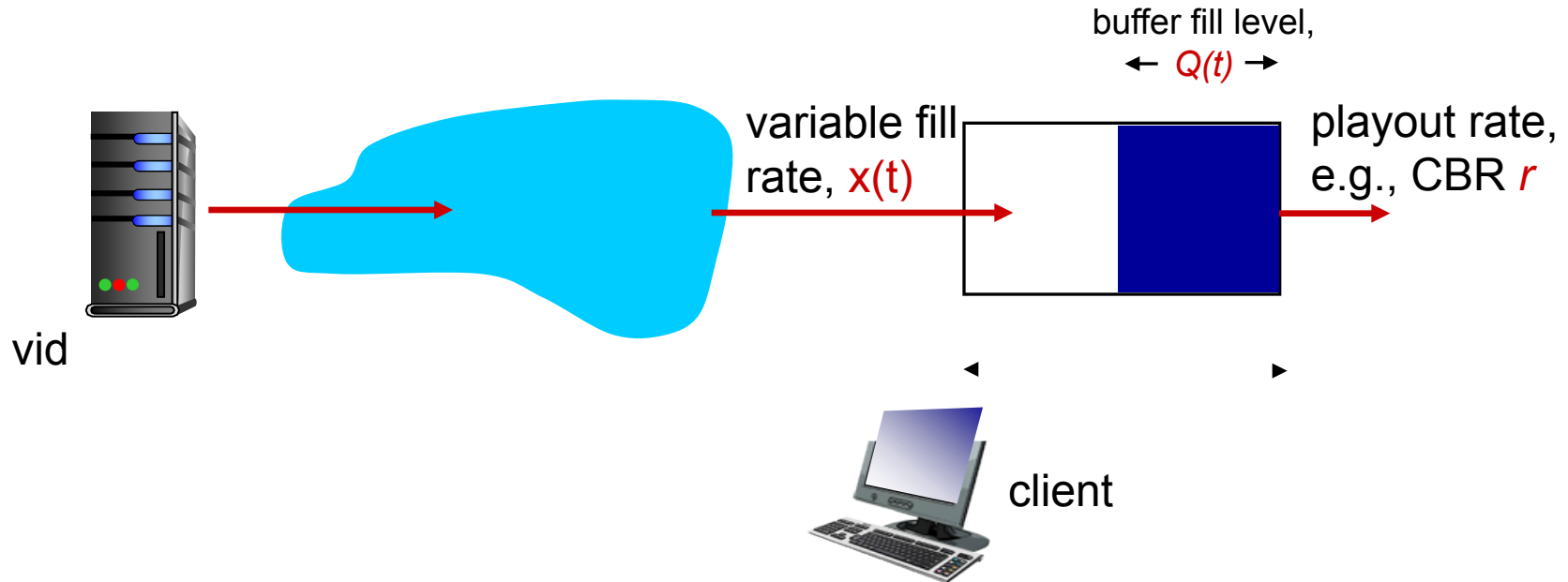
- ❖ other challenges:
 - client interactivity: pause, fast-forward, rewind, jump through video
 - video packets may be lost

Streaming stored video: ~~revisted~~

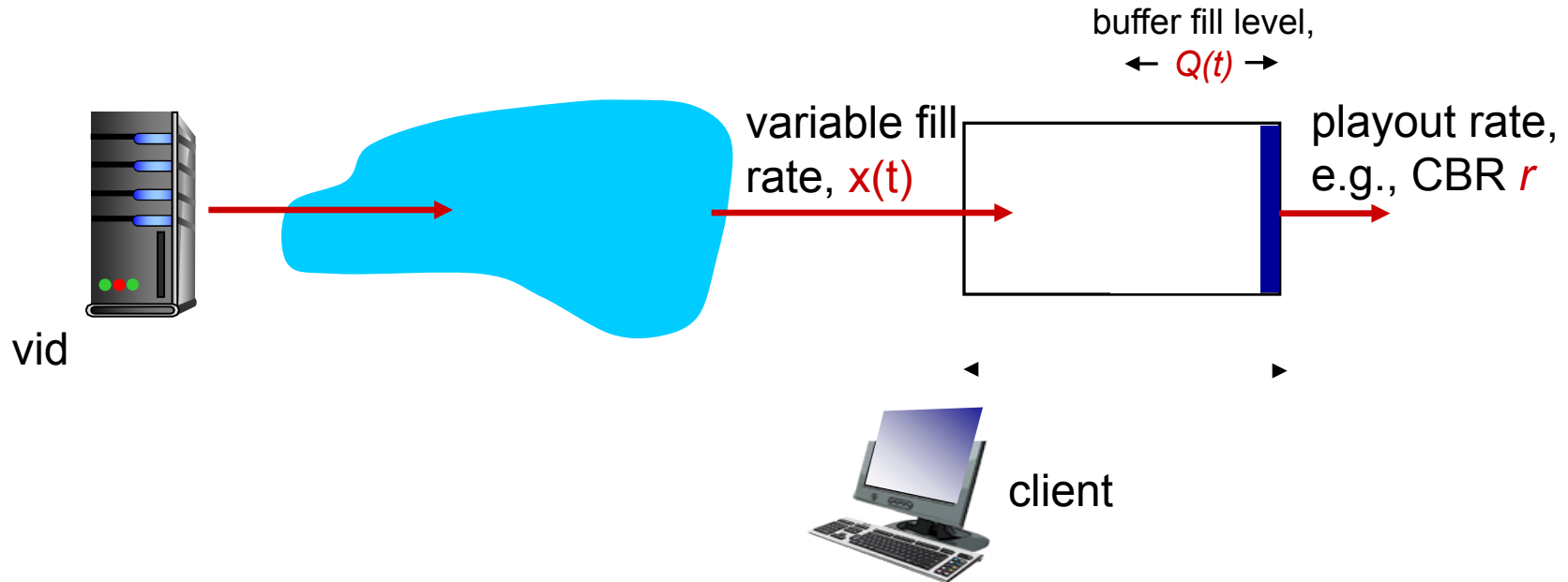


- ❖ *client-side buffering and playout delay:*
compensate for network-added delay,
delay jitter

Client-side buffering, playout

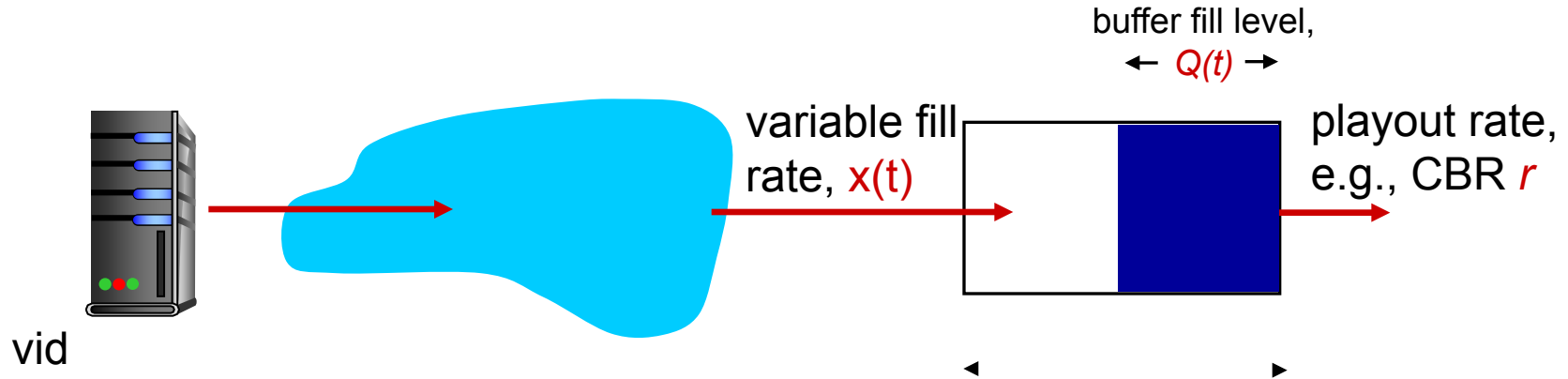


Client-side buffering, playout



1. Initial fill of buffer until playout begins at t_p
2. playout begins at t_p ,
3. buffer fill level varies over time as fill rate $x(t)$ varies and playout rate r is constant

Client-side buffering, playout



playout
playout rate (r):

- ❖ $x < r$: buffer eventually empties (causing freezing of video playout until buffer again fills)
- ❖ $x > r$: buffer will not empty, provided initial playout delay is large enough to absorb variability in $x(t)$
 - *initial playout delay tradeoff*: buffer starvation less likely with larger delay, but

Streaming multimedia: UDP

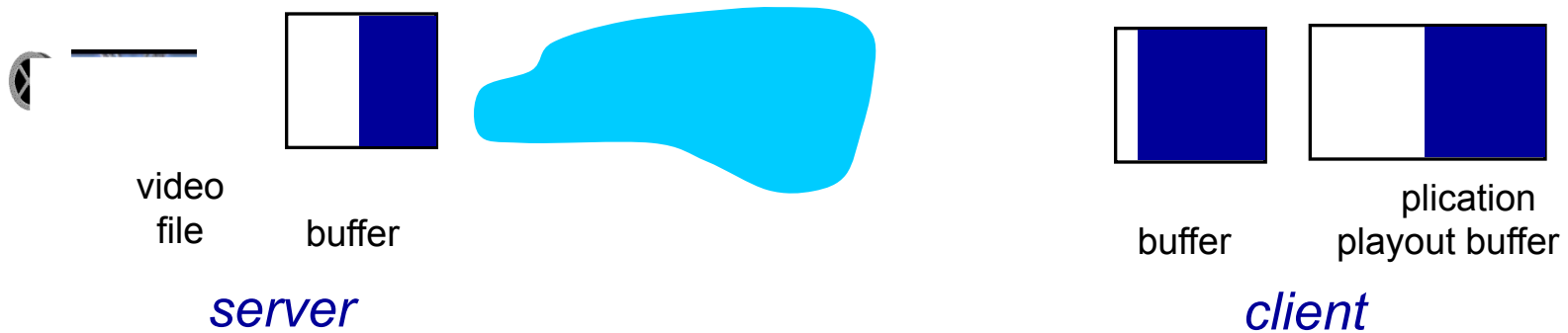
- ❖ server sends at rate appropriate for client
 - often: send rate = encoding rate = constant r

CO

- ❖ short playout delay (2-5 seconds) to remove network jitter
- ❖ error recovery: application-level, time permitting
- ❖ RTP [RFC 2326]: multimedia payload types
- ❖ UDP may *not* go through firewalls

Streaming multimedia: ~~HTTP~~

- ❖ multimedia file retrieved via HTTP GET
- ❖ send at maximum possible rate under TCP



- ❖ fill rate fluctuates due to TCP congestion control, retransmissions (in-order delivery)
- ❖ larger playout delay: smooth TCP delivery rate

Streaming multimedia:

~~DASH~~

❖ *DASH*: *D*ynamic, *A*daptive *S*teaming over *H*TTP

❖ *server*:

- divides video into small chunks

at different bit rates

- *manifest file*: provides URLs for different chunks

❖ *client*:

- periodically measures server-to-client bandwidth
- consulting manifest, requests one chunk at a time
 - chooses maximum coding rate sustainable given current bandwidth

Streaming multimedia:

~~DASH~~

- ❖ *DASH: Dynamic, Adaptive Streaming over HTTP*
- ❖ *"intelligence"* at client: client determines
 - *what encoding rate* to request (higher quality when more bandwidth available)
 - *where* to request chunk (can request from URL server that is "close" to client or has high available bandwidth)

Content distribution networks

❖ *challenge:* how to stream content (selected from millions of videos) to hundreds of thousands of simultaneous users?

❖ *optio*

r''

- single point of failure
- point of network congestion
- long path to distant clients
- multiple copies of video sent over outgoing link

....quite simply: this solution *doesn't scale*

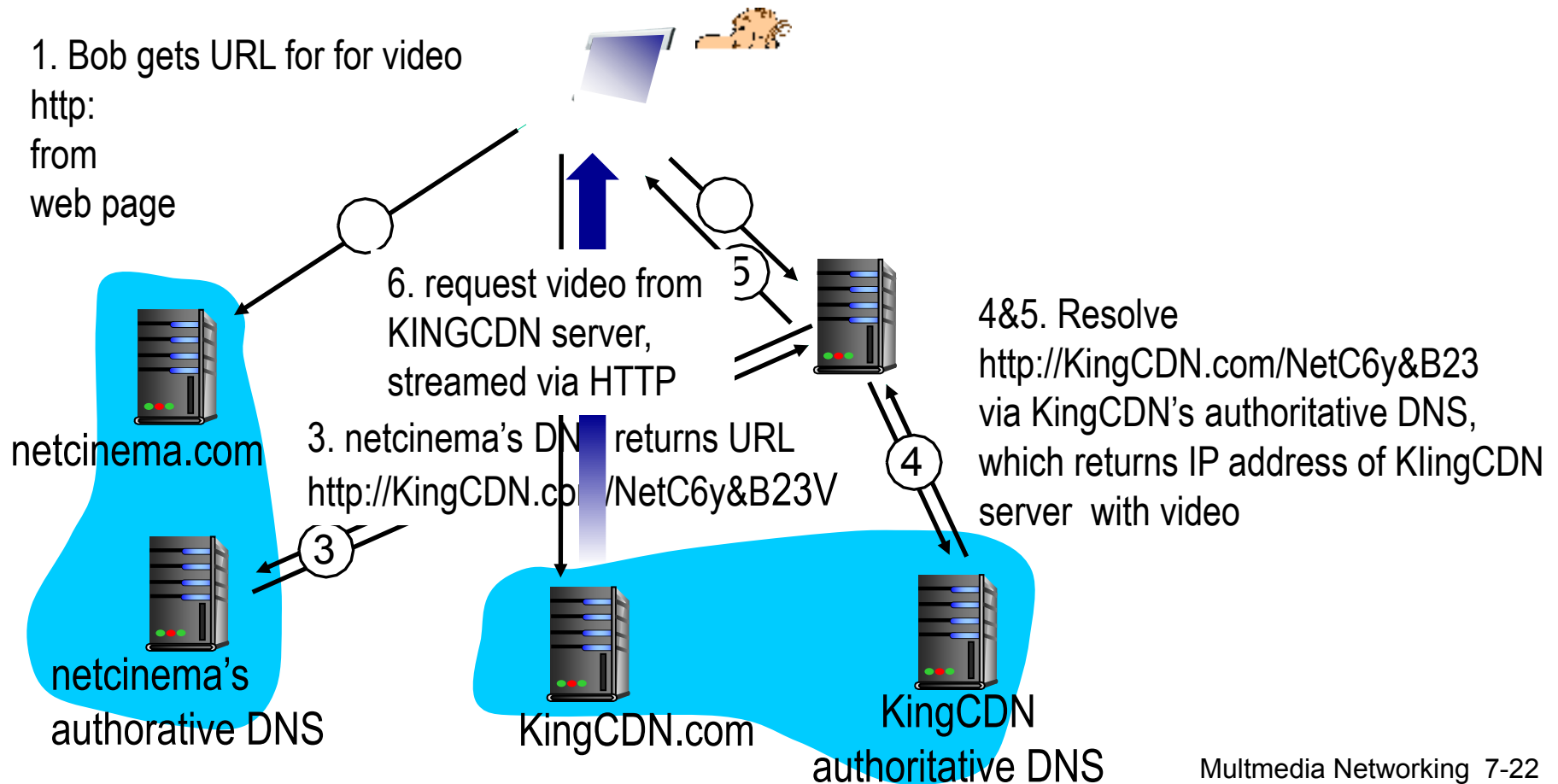
Content distribution networks

- ❖ *challenge*: how to stream content (selected from millions of videos) to hundreds of thousands of simultaneous users?
- ❖ *options* of videos at multiple geographically distributed sites (*CDN*)
 - *enter deep*: push CDN servers deep into many access networks
 - close to users
 - used by Akamai, 1700 locations
 - *bring home*: smaller number (10's) of larger clusters in POPs near (but not within) access networks

CDN: “simple” content access scenario

Bob (client) requests video <http://netcinema.com/>

- video stored in CDN at <http://KingCDN.com/NetC6y8>



CDN cluster selection strategy

- ❖ *challenge*: how does CDN DNS select “good” CDN node to stream to client
 - pick CDN node geographically closest to client

pin
(DNS)

CDN

- IP anycast

- ❖ *alternative*: let *client* decide - give client a list of several CDN servers
 - client pings servers, picks “best”
 - Netflix approach

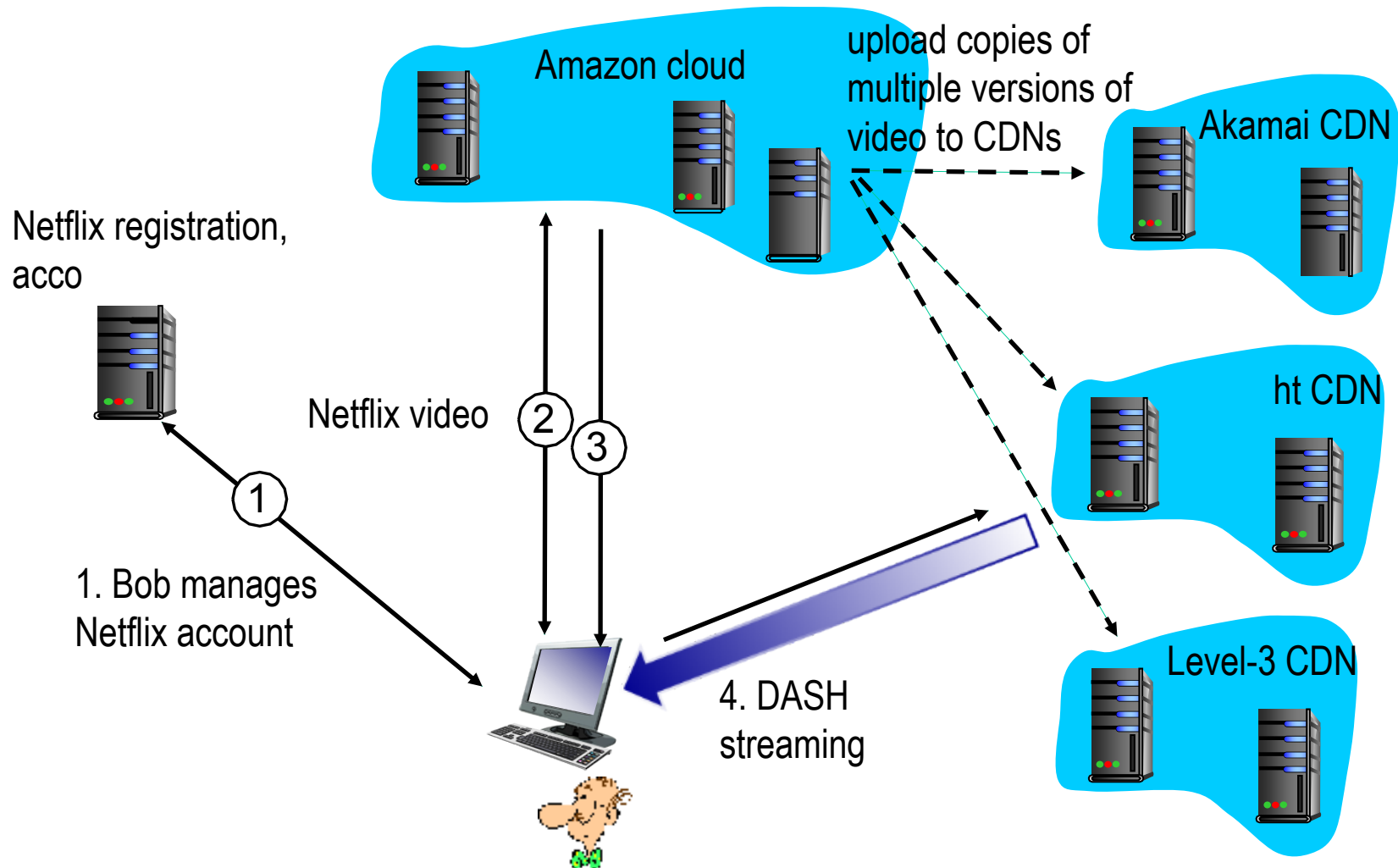
Case study: Netflix

- ❖ 30% downstream US traffic in 2011
- ❖ owns very little infrastructure, uses 3rd party services:

- amazon
cloud
 - create multiple version of movie (different encodings) in cloud
 - upload versions from cloud to CDNs
 - Cloud hosts Netflix web pages for user browsing

- *three* 3rd party CDNs host/stream

Case study: Netflix



Multimedia networking: ~~outline~~

7.1 multimedia networking
applications

7.2 streamin 0

7.4 pr
conversational applications

7.5 network support for multimedia

Voice-over-IP (VoIP)

- ❖ *VoIP end-end-delay requirement:* needed to maintain “conversational” aspect
 - higher delay air interactivity
 - > 4
 - includes application-level (packetization, playout), network delays
- ❖ *session initialization:* how does callee advertise IP address, port number, encoding algorithms?
- ❖ *value-added services:* call forwarding, screening, recording
- ❖ *emergency services:* 911

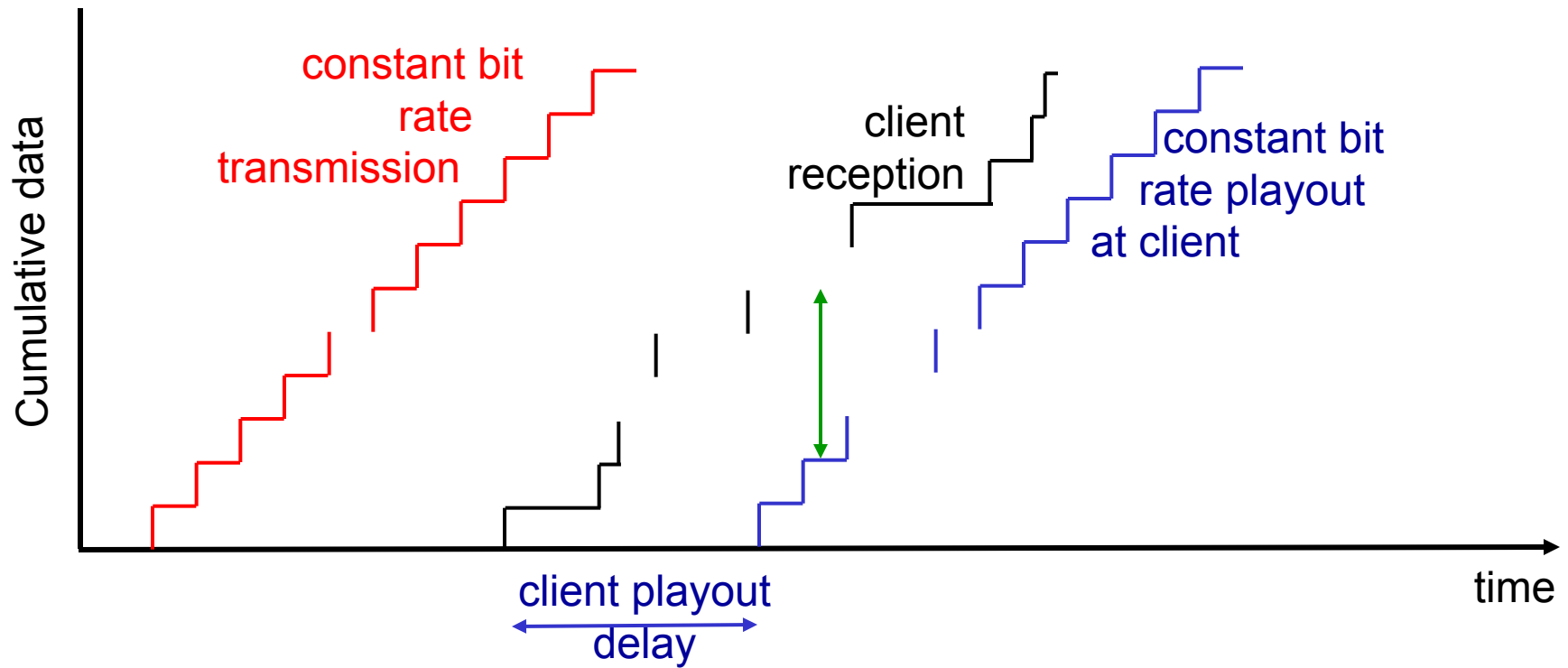
VoIP characteristics

- ❖ speaker's audio: alternating talk spurts, silent periods.
 - 64 kbps during talk spurt
 - pkts genera talk spurts
- of
- ❖ application-layer header added to each chunk
- ❖ chunk+header encapsulated into UDP or TCP segment
- ❖ application sends segment into socket every 20 msec during talkspurt

VoIP: packet loss, delay

- ❖ *network loss*: IP datagram lost due to network congestion (router buffer overflow)
- ❖ *delay loss*: I gives too late en
 - typical maximum tolerable delay: 400 ms
- ❖ *loss tolerance*: depending on voice encoding, loss concealment, packet loss rates between 1% and 10% can be tolerated

Delay jitter



- ❖ end-to-end delays of two consecutive packets: difference can be more or less than 20 msec (transmission time difference)

VoIP: fixed playout delay

- ❖ receiver attempts to playout each chunk exactly q msecs after chunk was generated.

- chunk ha : play out

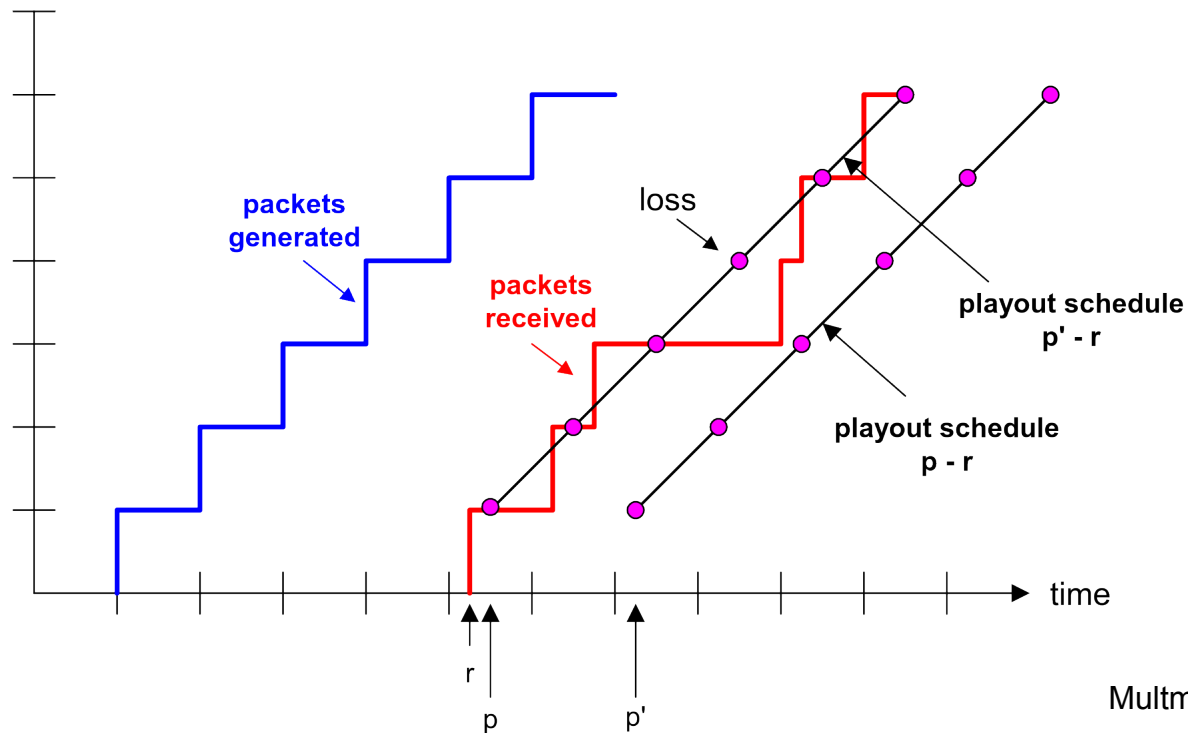
t

- ❖ tradeoff in choosing q :
 - *large q : less packet loss*
 - *small q : better interactive experience*

VoIP: fixed playout delay

- sender generates packets every 20 msec during ta
- first packet received at time r
- first playout schedule: begins at p
- second playout schedule: begins at p'

packets



Adaptive playout delay (1)

- ❖ *goal*: low playout delay, low late loss rate
- ❖ *approach*: adaptive playout delay adjustment:

- silence
- chunks still played out every 20 msec during talk spurt

- ❖ adaptively estimate packet delay:
(EWMA - exponentially weighted moving average)

delay estimate
after i th packet

small constant,
e.g. 0.1

$$d_i = (1 - \alpha)d_{i-1} + \alpha(r_i - t_i)$$

time received

time sent

(timestamp)

measured delay of i th packet

Adaptive playout delay (2)

- ❖ also useful to estimate average deviation of c

$$v_i = (1-\beta)v_{i-1} + \beta |r_i - t_i - d_i|$$

start

- ❖ for first packet in talk spurt, playout time is: $playout-time_i = t_i + d_i + Kv_i$

remaining packets in talkspurt are
played out periodically

Adaptive playout delay (3)

Q: How does receiver determine whether packet is first in a talkspurt?

- ❖ if no loss, receiver looks at successive timestamps

- ❖ with ok at
both time stamps and sequence numbers
 - difference of successive stamps > 20 msec
and sequence numbers without gaps --> talk spurt begins.

VoIP: recovery from packet loss (1)

Challenge: recover from packet loss given small tolerable delay between original transmission and playout

- ❖ each ACK/NAK

retr
in Ch. 5)

parity

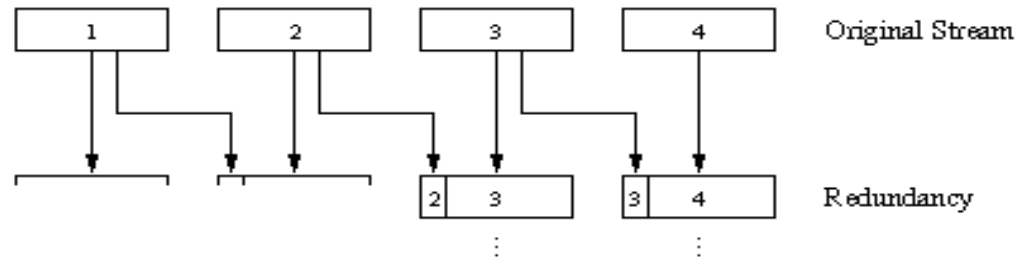
simple FEC

- ❖ for every group of n chunks, create redundant chunk by exclusive OR-ing n original chunks
- ❖ send $n+1$ chunks, increasing bandwidth by factor $1/n$
- ❖ can reconstruct original n chunks if at most one lost chunk from $n+1$ chunks, with playout delay

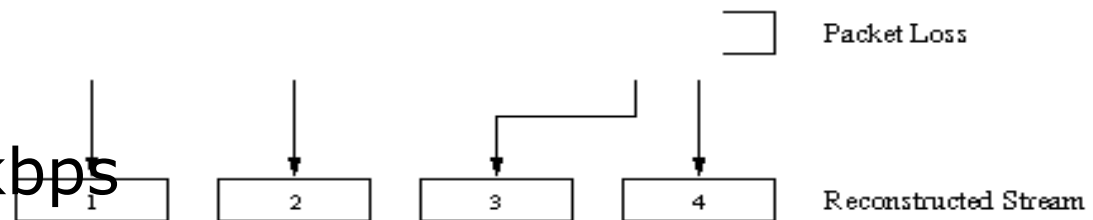
VoIP: recovery from packet loss (2)

another FEC scheme:

- ❖ “piggyback lower quality stream”
- ❖ send lower resol

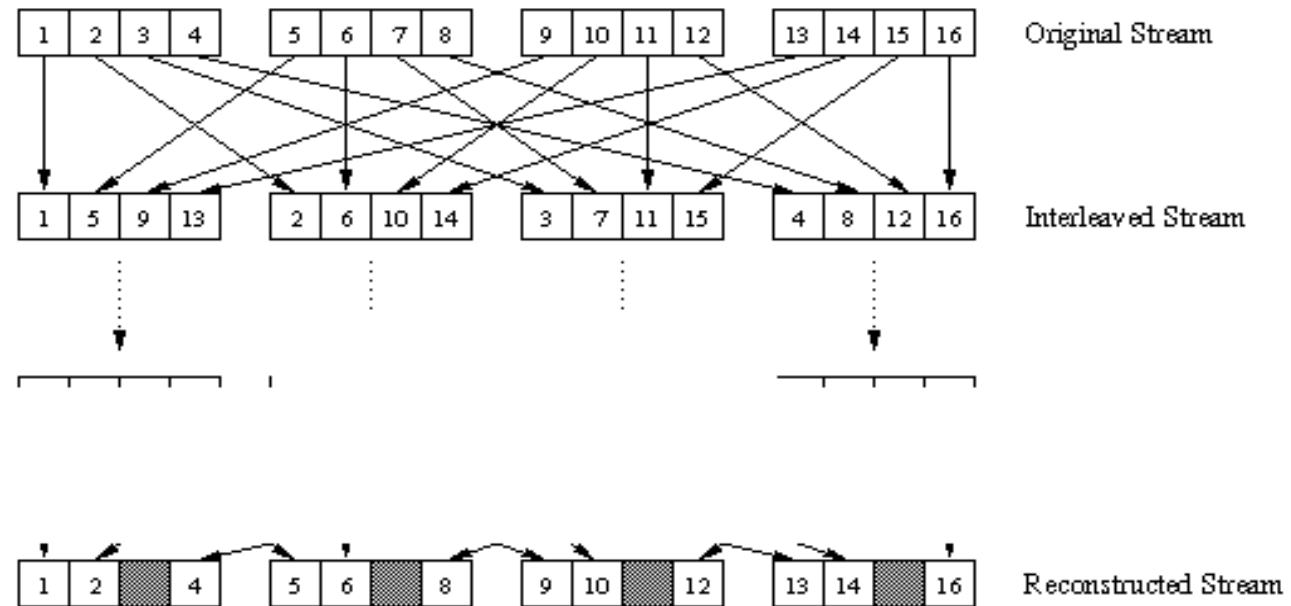


- ❖ e.g., n
stream PCM at 64 kbps
and redundant stream
GSM at 13 kbps



- ❖ non-consecutive loss: receiver can conceal loss
- ❖ generalization: can also append (n-1)st and (n-2)nd chunk

VoIP: recovery from packet loss (3)



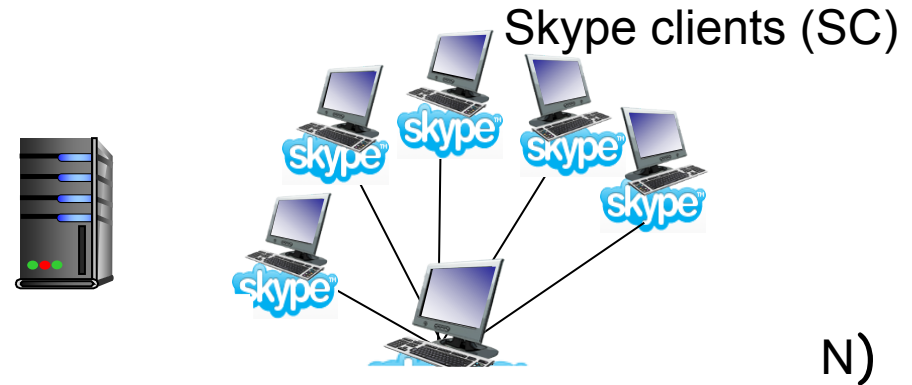
interleaving to conceal loss:

- ❖ audio chunks divided into smaller units, e.g. four 5 msec units per 20 msec audio chunk
- ❖ packet contains small units from different

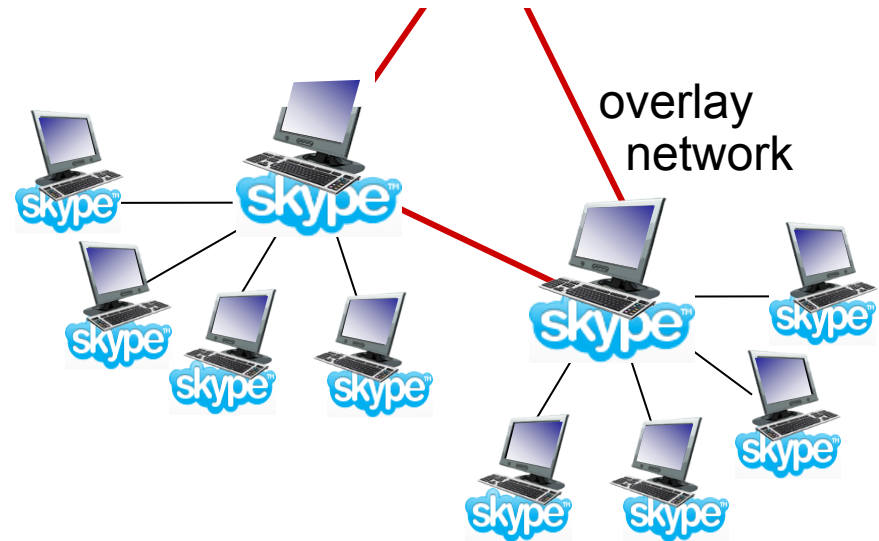
- ❖ if packet lost, still have *most* of every original chunk
- ❖ no redundancy overhead, but increases playout delay

Voice-over-IP: Skype

- ❖ proprietary application-layer protocol (inferred via reverse engineering)



- ❖
 - direct other for VoIP call
 - **super nodes (SN):** skype peers with special functions
 - **overlay network:** among SNs to locate SCs
 - **login server**



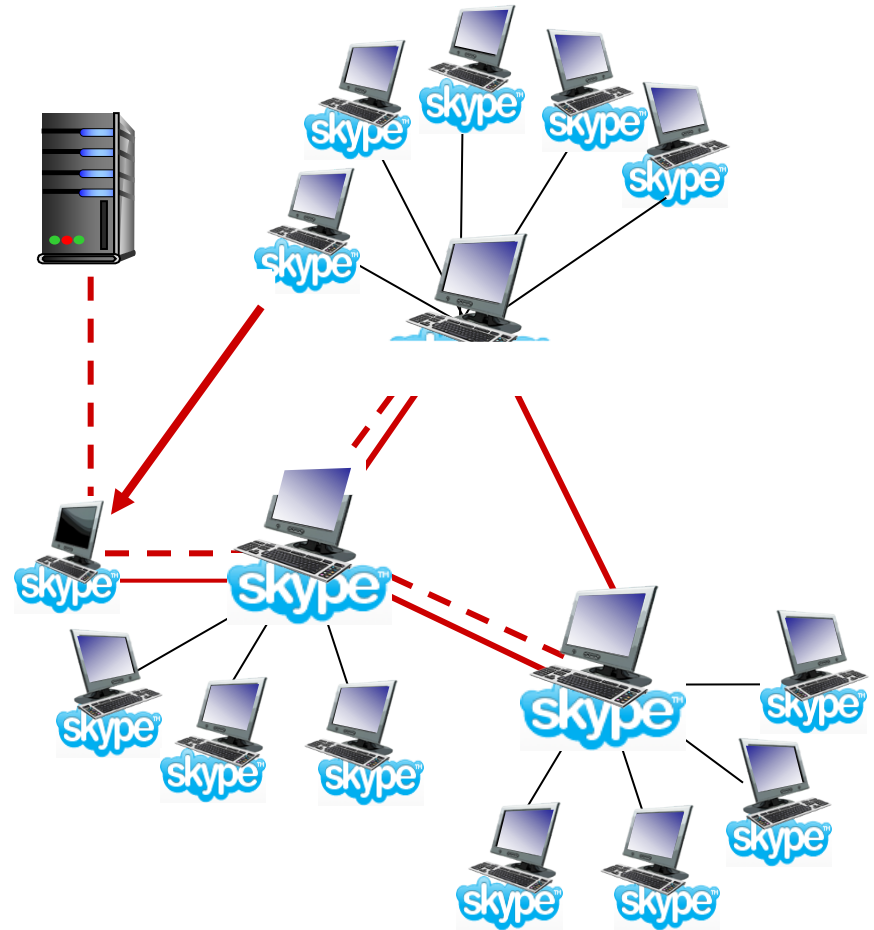
P2P voice-over-IP: skype

skype client

operation:

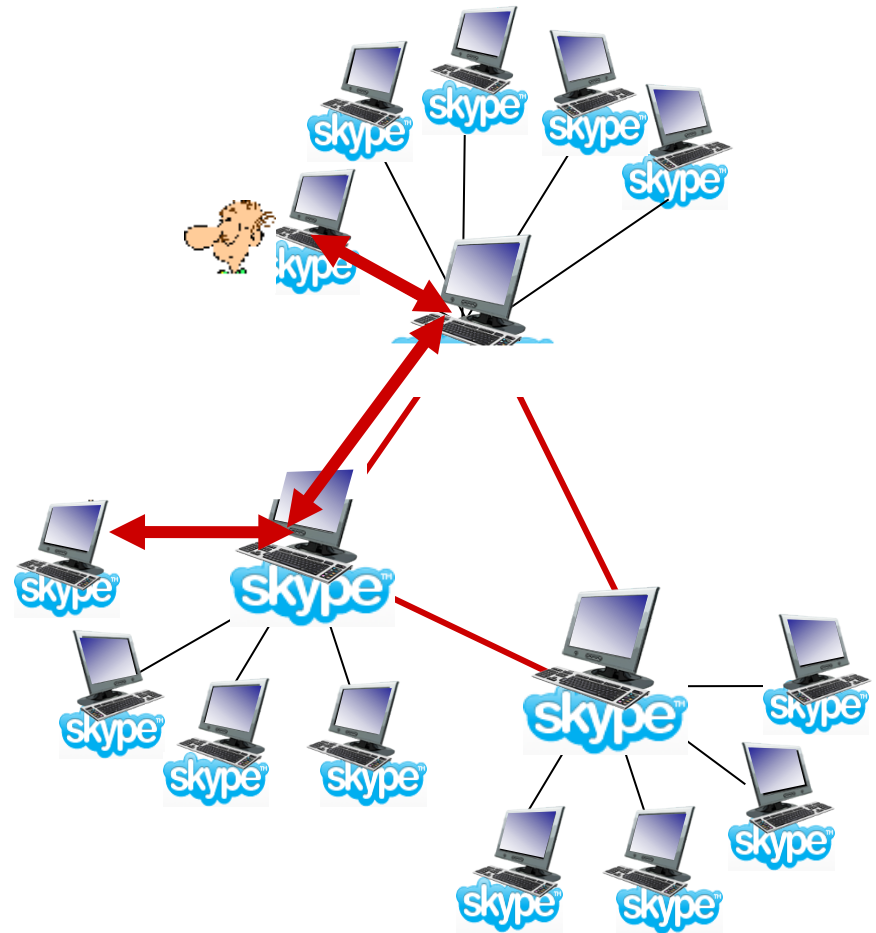
1. joins skype network by contacting SN (I

- central login server
3. obtains IP address for callee from SN, SN overlay
 - or client buddy list
4. initiate call directly to callee



Skype: peers as relays

- ❖ *problem:* both Alice, Bob are behind "NATs"
 - NAT prevents outside peer from initiating connection to inside
- ❖ *relays*
 - *Bob maintain open connection to their SNs*
 - Alice signals her SN to connect to Bob
 - Alice's SN connects to Bob's SN
 - Bob's SN connects to Bob over open connection Bob initially initiated to his SN



Multimedia networking: ~~outline~~

7.1 multimedia networking
applications

7.2 streamin o

7.4 pr
conversational applications:
RTP, SIP

7.5 network support for multimedia

Real-Time Protocol (RTP)

- ❖ RTP specifies packet structure for packets carrying audio

- ❖ RTP runs in end systems

- ❖ RTP packets are encapsulated in

- ❖

- ❖ RTP packet provides
 - payload type identification
 - packet sequence numbering
 - time stamping

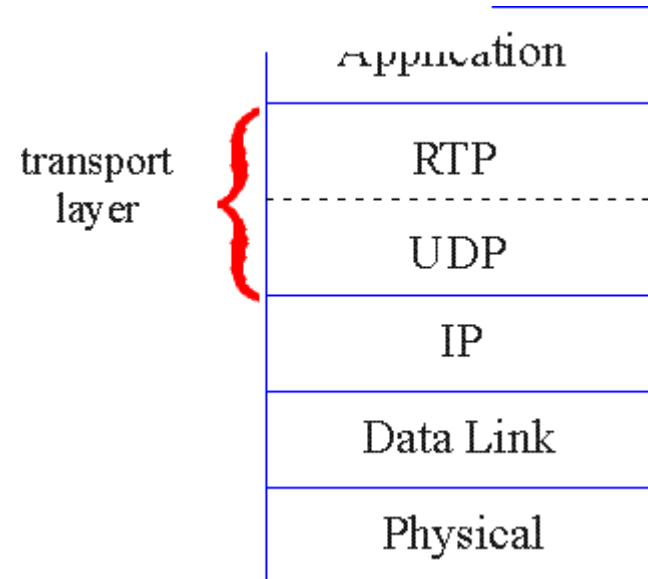
typical: if two VoIP applications run RTP, they may be able to work together

RTP runs on top of UDP

RTP libraries provide transport-layer interface that extends UDP:

- port numbers, IP addresses

- ti



RTP example

example: sending 64 kbps PCM-encoded voice over RTP

- ❖ application c

20

bytes in a chunk

- ❖ audio chunk + RTP header form RTP packet, which is encapsulated in UDP segment

- ❖ RTP header indicates type of audio encoding in ch packet

ference

- ❖ RTP header also contains sequence numbers, timestamps

RTP and QoS

- ❖ RTP does *not* provide any mechanism to ensure timely data delivery or other QoS guarantees
- ❖ RTP encapsulation at end
 ■ router, making no special effort to ensure that RTP packets arrive at destination in timely manner

RTP header

<i>payload type</i>	<i>sequence number</i>	<i>time stamp</i>	<i>Synchronization Source ID</i>	<i>Miscellaneous fields</i>
-------------------------	----------------------------	-------------------	--------------------------------------	---------------------------------

payload type (7 bits): indicates type of encoding currently being

er

Paylo

Payload type 3: GSM, 13 kbps

Payload type 7: LPC, 2.4 kbps

Payload type 26: Motion JPEG

Payload type 31: H.261

Payload type 33: MPEG2 video

sequence # (16 bits): increment by one for each RTP packet sent

❖ detect packet loss, restore packet sequence

RTP header

payload type	sequence number	time stamp	Synchronization Source ID	Miscellaneous fields
-----------------	--------------------	------------	------------------------------	-------------------------

- ❖ *timestamp field (32 bits long)*: sampling instant of first RTP data

-

for 25
µsecs for 8 KHz sampling clock)

- if application generates chunks of 160 encoded samples, timestamp increases by 160 for each RTP packet when source is active. Timestamp clock continues to increase at constant rate when source is inactive.

- ❖ *SSRC field (32 bits long)*: identifies source

RTSP/RTP programming ~~assignment~~

- ❖ build a server that encapsulates stored video frames into RTP packets
 - grab video frames, create headers, create RTP packets
 - include `librtsp` and `librtp`
 - client RTP provided for you
- ❖ also write client side of RTSP
 - issue play/pause commands
 - server RTSP provided for you

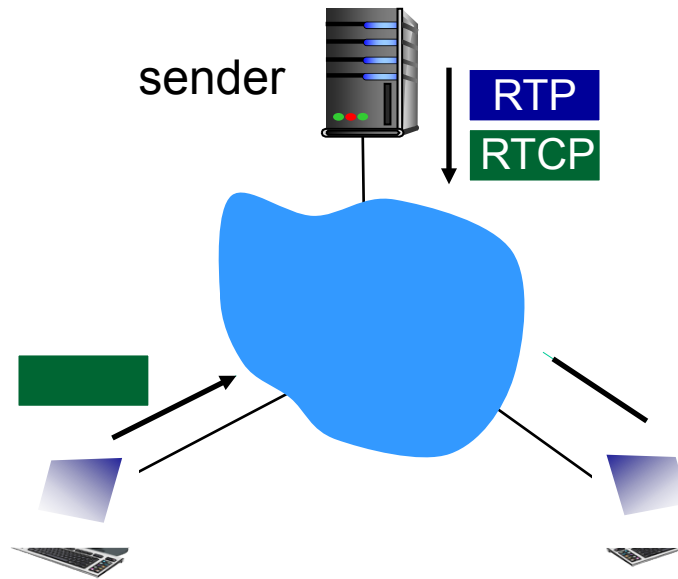
Real-Time Control Protocol (RTCP)

- ❖ works in conjunction with RTP
- ❖ each participant
- ❖ each RTCP packet contains sender and/or receiver rts

RTCP
packets to all other
participants

- #
packets sent, #
packets lost,
interarrival jitter
- ❖ feedback used to control performance
 - sender may modify its transmissions based on feedback

RTCP: multiple multicast senders



- ❖ each RTP session: typically a single multicast address; all RTP /RTCP packets belonging to session use multicast address
- ❖ RTP, RTCP packets distinguished from each other via distinct port numbers
- ❖ to limit traffic, each participant reduces RTCP traffic as number of conference participants increases

RTCP: packet types

receiver report packets:

- ❖ fraction of packets lost, last sequence number

source description packets:

- ❖ e-mail address of sender, sender's name

sender packets:

- ❖ SSRC of RTP stream, current time, number of packets sent, number of bytes sent

- ❖ provide mapping between the SSRC and the user/host name

RTCP: stream ~~synchronization~~

- ❖ RTCP can synchronize different media streams within a RTP session

generally one RTP packet per media stream for video, one for audio.

- ❖ timestamps in RTP packets tied to the video, audio sampling clocks
 - *not* tied to wall-clock time

- ❖ each RTCP sender-report packet contains (for most commonly generated

for RTP

packet

- wall-clock time for when packet was created
- ❖ receivers use association to synchronize playout of audio, video

RTCP: bandwidth scaling

RTCP attempts to limit its traffic to 5% of session bandwidth

- ❖ 75 kbps is equally shared among receivers:

R receivers, each

sendi
Mbps

- ❖ RTCP attempts to limit RTCP traffic to 100 Kbps
- ❖ RTCP gives 75% of rate to receivers; remaining 25% to sender

- ❖ sender gets to send RTCP traffic at 25 kbps.
- ❖ participant determines RTCP packet transmission period by calculating avg RTCP packet size (across entire session) and dividing by allocated rate

SIP: Session Initiation Protocol

[RFC 3261]

long-term vision:

- ❖ all telephone calls, video conference calls take place over Internet
- ❖ people identify themselves or e-mail addresses
- ❖ can roam anywhere (roaming), no matter where callee roams, no matter what IP device callee is currently using

SIP services

- ❖ SIP provides mechanisms for call setup:

- for caller to

est

- so caller, callee can agree on media type, encoding
- to end call

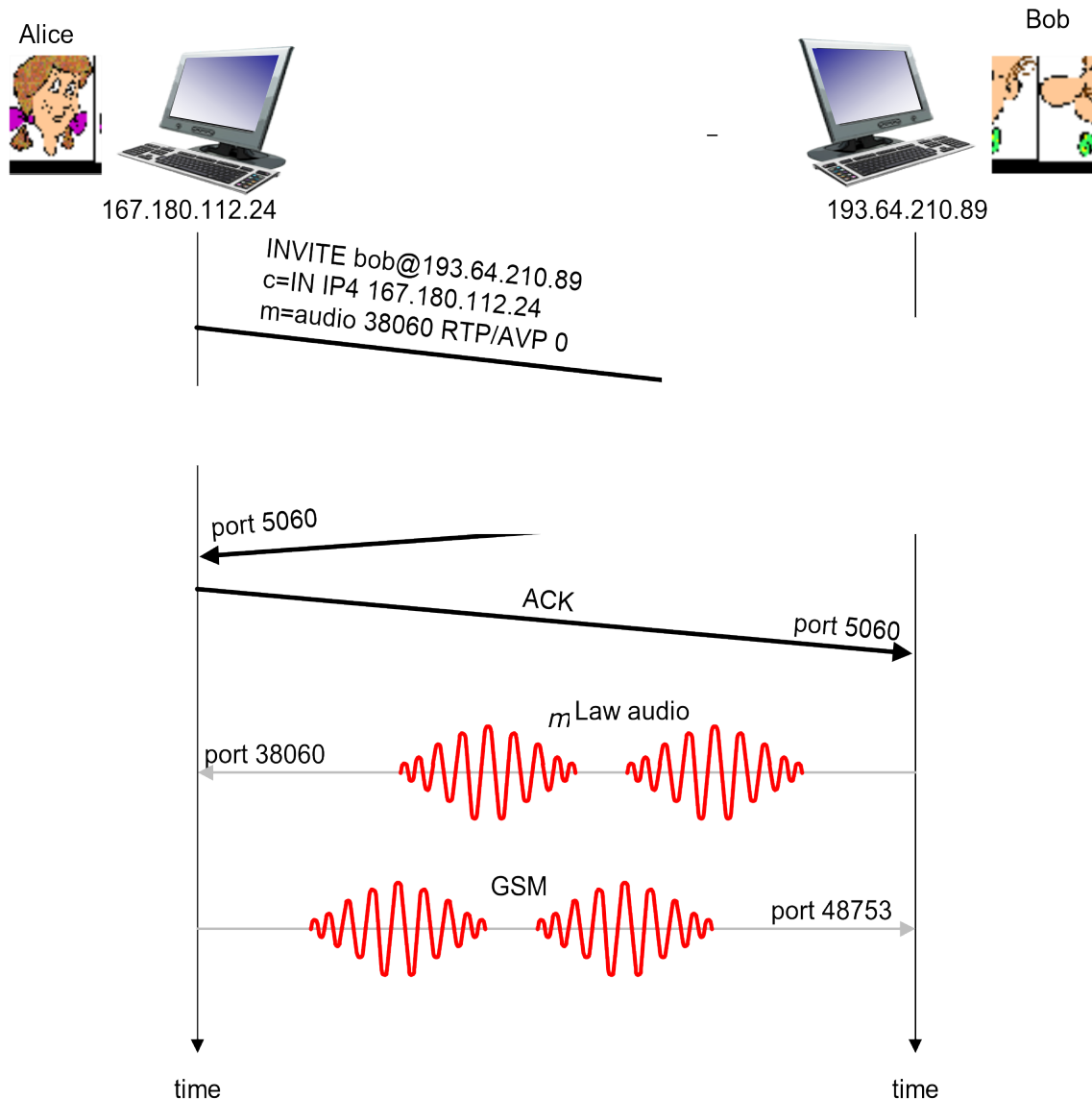
- ❖ determine current IP address of callee:

ps mnemonic

ent:

- add new media streams during call
- change encoding during call
- invite others
- transfer, hold calls

Example: setting up call to known IP address



❖ Alice's SIP invite message indicates her port number, IP address, encoding she prefers to

OK message indicates his port number, IP address, preferred encoding (GSM)

❖ SIP messages can be sent over TCP or UDP; here sent over RTP/UDP

❖ default SIP port is 5060

Setting up a call (more)

- ❖ codec negotiation:
 - suppose Bob doesn't have PCM μ law encoder
- ❖ rejecting a call
 - Bob can reject with replies "usy," "gone,"

Acc
listing his encoders.
Alice can then send
new INVITE
message,
advertising different
encoder

- ❖ media can be sent over RTP or some other protocol

Example of SIP message

```
INVITE sip:bob@domain.com SIP/2.0
Via: SIP/2.0/UDP 167.180.112.24
From: sip:alice@hereway.com
To: sip:bob@domain.com
Call-ID: a2e3a@pig
```

```
c=IN IP4 167.180.112.24
m=audio 38060 RTP/AVP 0
```

❖ Here we don't know Bob's IP address

~~SIP~~ Speeded
messages using
SIP default port
506

Notes:

- ❖ HTTP message syntax
- ❖ sdp = session description protocol
- ❖ Call-ID is unique for every call

❖ Alice specifies in header that SIP client sends, receives

Name translation, user location

- ❖ caller wants to call callee, but only has callee's name or e-mail address.
- ❖ result can be based on:
 - time of day (work, e)

current

- user moves around
- DHCP protocol
- user has different IP devices (PC, smartphone, car device)
- status of callee (calls sent to voicemail when callee is already talking to someone)

SIP registrar

- ❖ one function of SIP server: *registrar*
- ❖ when Bob starts SIP client, client sends SIP REGISTER message to Bob's registrar

r

```
REGISTER
Via: SIP/2.0/UDP 193.64.210.89
From: sip:bob@domain.com
To: sip:bob@domain.com
Expires: 3600
```

SIP proxy

- ❖ another function of SIP server: *proxy*
- ❖ Alice sends invite message to her proxy server

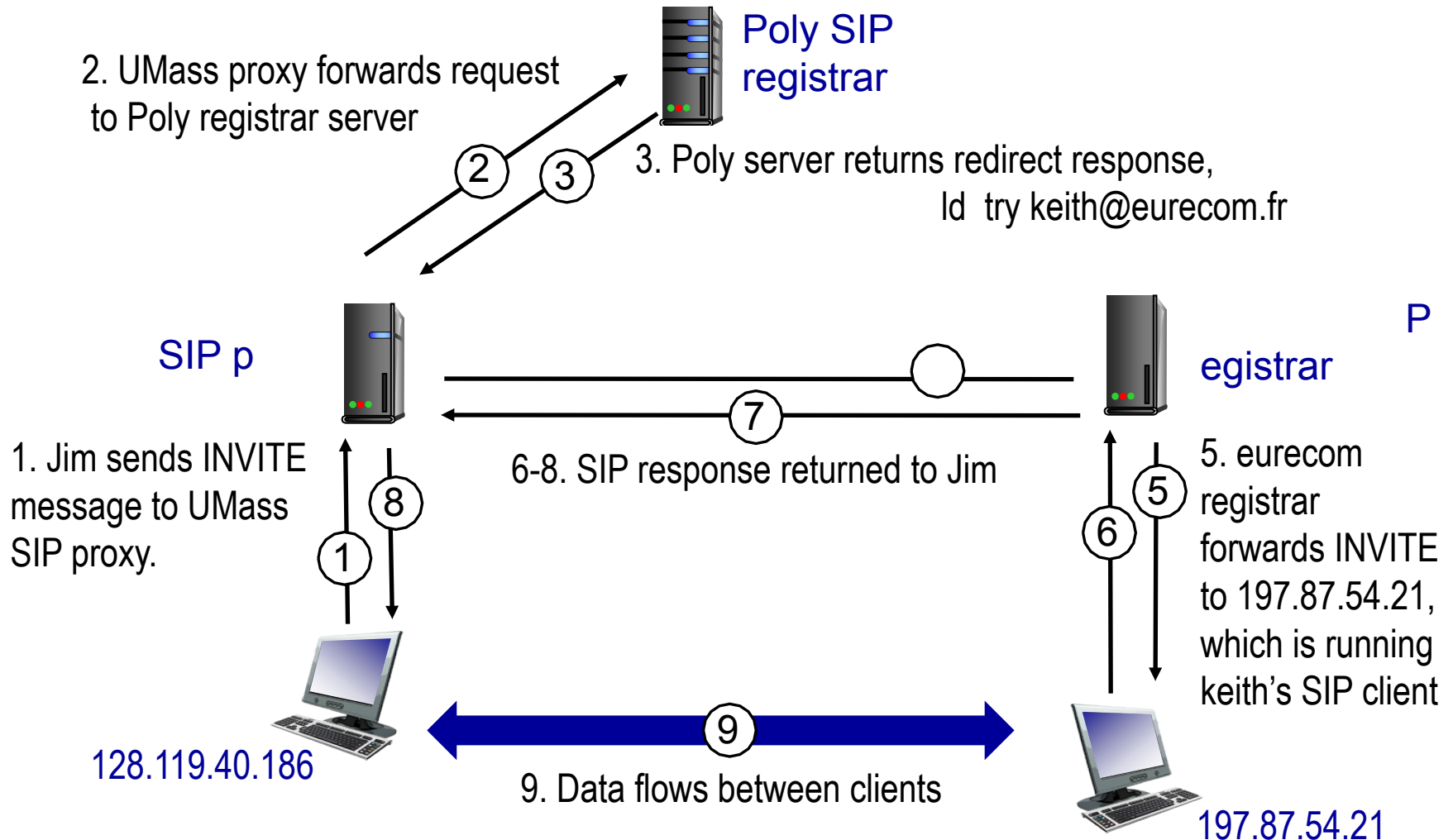
call

s

- ❖ Bob sends response back through same set of SIP proxies
- ❖ proxy returns Bob's SIP response message to Alice
 - contains Bob's IP address
- ❖ SIP proxy analogous to local DNS server plus TCP setup

SIP example: jim@umass.edu calls

~~keith@poly.edu~~



Comparison with H.323

- ❖ H.323: another signaling protocol for real-time, interactive multimedia

- ❖ H.323 comes from the ITU (telephony)
- ❖ SIP borrows much from H.323

suite
multimedia
conferencing:
signaling,
registration,
admission control,
transport, codecs

- ❖ SIP: single component. Works with RTP, but does not encapsulate it. Can

flavor; H.323 has telephony flavor

- ❖ SIP uses KISS principle: **K**ee**P** **I**t **S**imple **S**tupid

Multimedia networking: ~~outline~~

7.1 multimedia networking
applications

7.2 streamin o

7.4 pr
conversational applications

7.5 network support for multimedia

Network support for multimedia

Approach	Granularity	Guarantee	Mechanisms	Complex	Deployed?
Making best of best effort service	All traffic treated equally	None or soft	No network support	low	everywhere
Differentiated service			policing.		
Per-connection QoS	Per-connection flow	Soft or hard after flow admitted	Packet marking, scheduling, policing, call admission	high	little to none

Dimensioning best effort networks

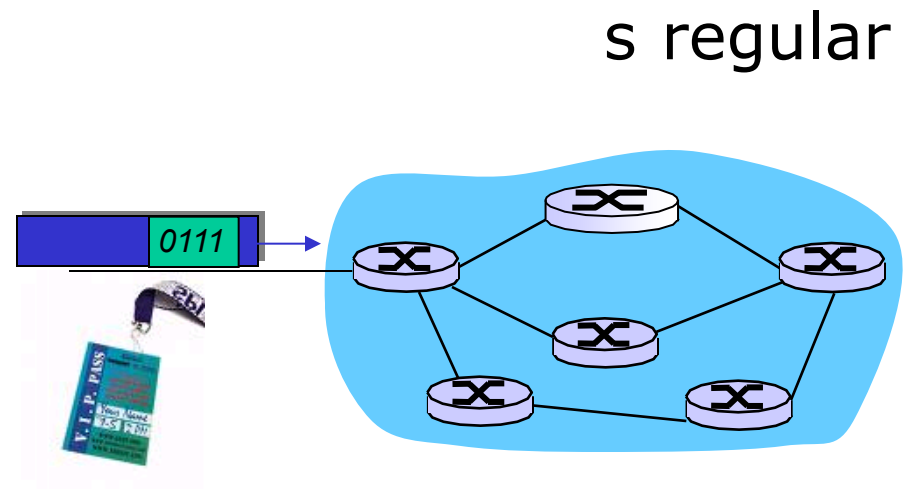
- ❖ *approach*: deploy enough link capacity so that congestion doesn't occur, multimedia traffic flows r loss
 - hig
- ❖ challenges:
 - *network dimensioning*: how much bandwidth is "enough?"
 - *estimating network traffic demand*: needed to determine how much bandwidth is "enough" (for that much traffic)

Providing multiple classes of ~~service~~

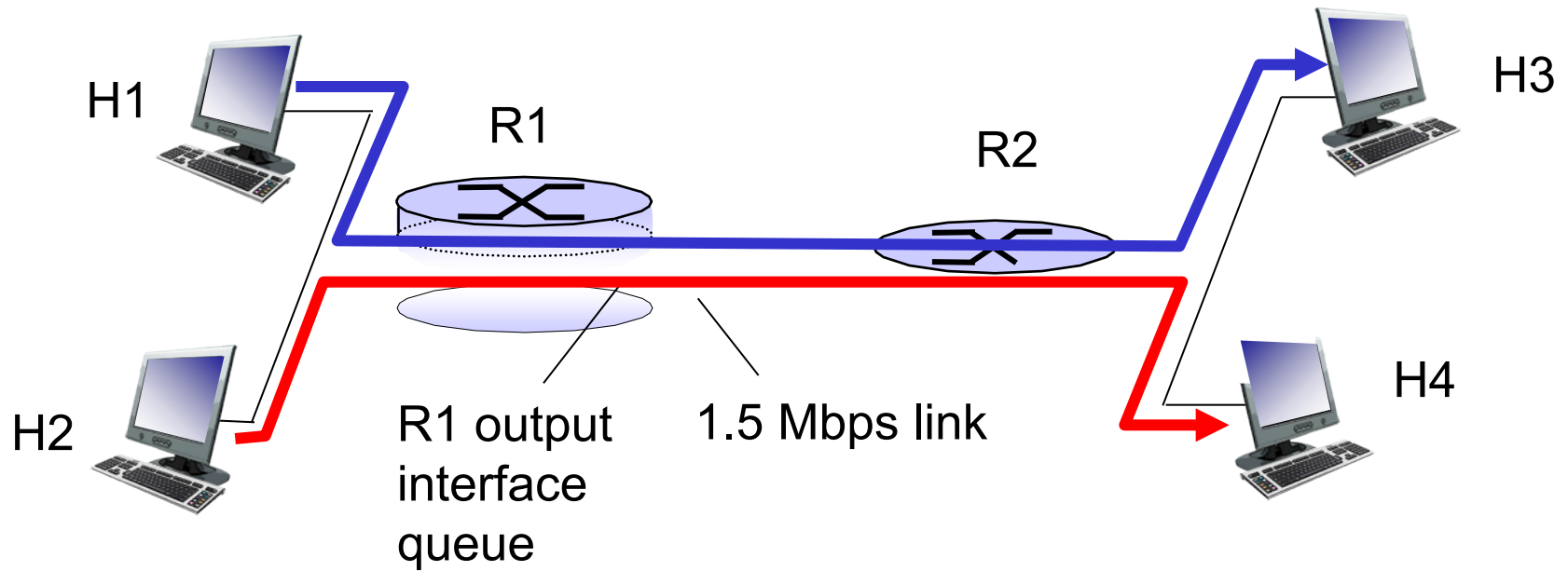
- ❖ thus far: making the best of best effort service
 - one-size fits all service model
- ❖ alternative: s of service

- ❖ granularity: differential service among multiple classes, *not among individual connections*

- ❖ history: ToS bits

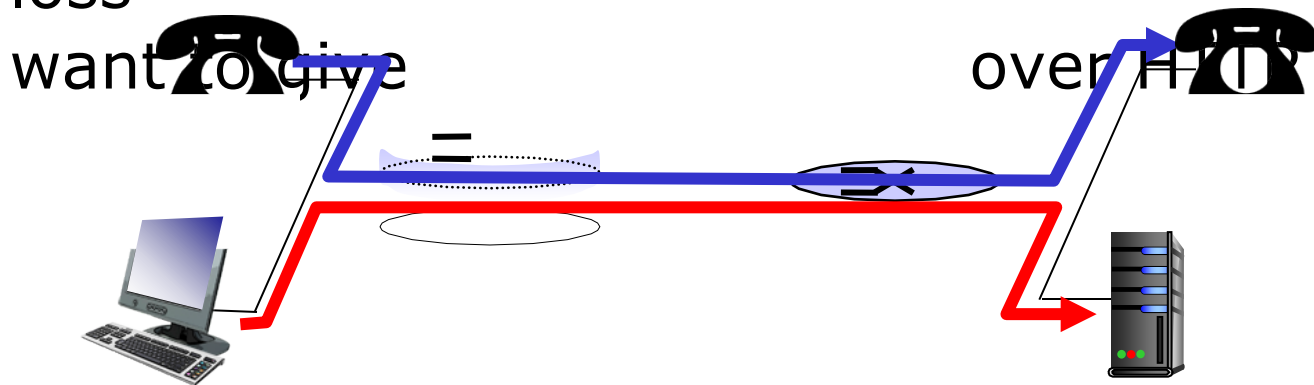


Multiple classes of service: scenario



Scenario 1: mixed HTTP and VoIP

- ❖ example: 1Mbps VoIP, HTTP share 1.5 Mbps link.
 - HTTP bursts can congest router, cause audio loss

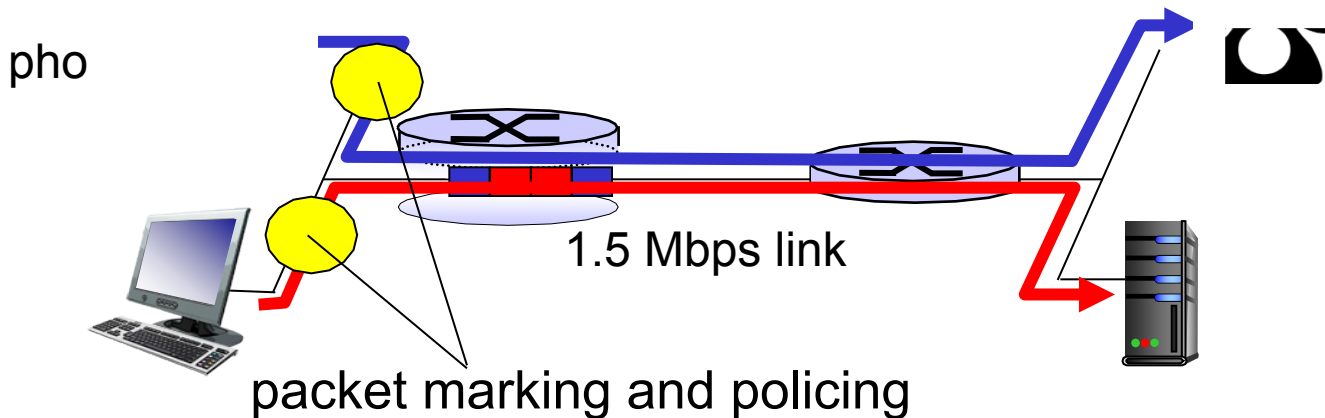


Principle 1

packet marking needed for router to distinguish between different classes; and new router policy to treat packets accordingly

Principles for QoS guarantees (more)

- ❖ what if applications misbehave (VoIP sends higher than declared rate)
 - policing: force source adherence to bandwidth allocations
- ❖ *marking, policing*

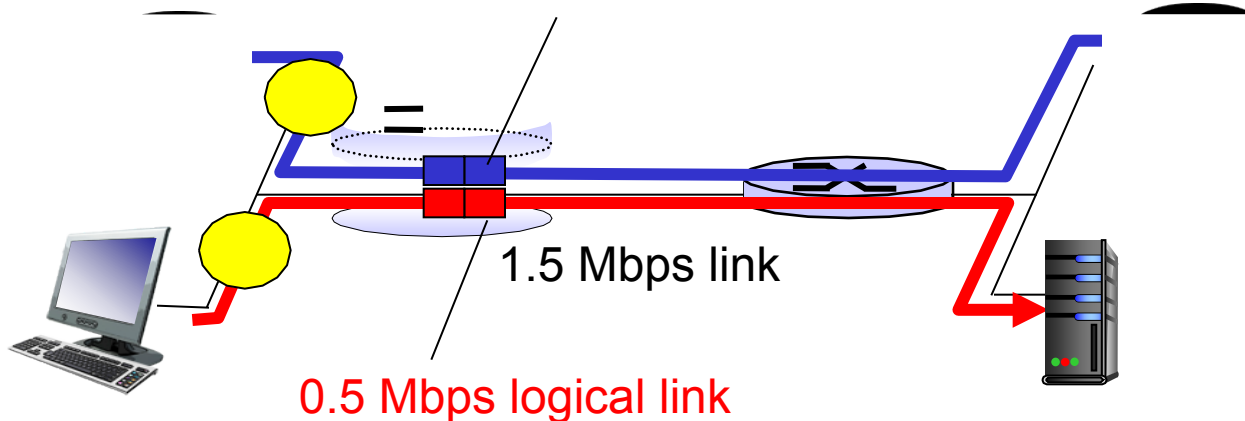


Principle 2

provide protection (isolation) for one class from others

Principles for QOS guarantees (more)

- ❖ allocating *fixed* (non-sharable) bandwidth to flow: *inefficient* use of bandwidth if flows doesn't use its allocation



Principle 3

while providing isolation, it is desirable to use resources as efficiently as possible

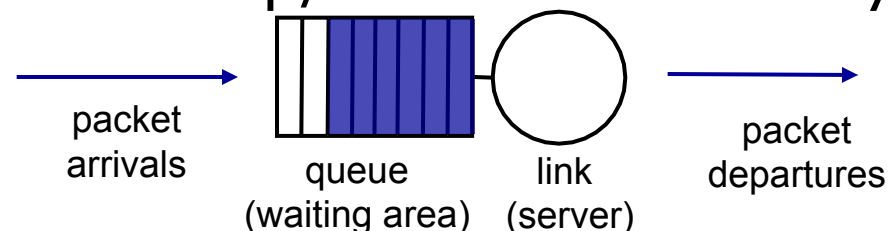
Scheduling and policing mechanisms

- ❖ *scheduling*: choose next packet to send on link
- ❖ *FIFO (first in first out) scheduling*: send in order of arriv

- *dis*
wh

queue:

- *tail drop*: drop arriving packet
- *priority*: drop/remove on priority basis
- *random*: drop/remove randomly

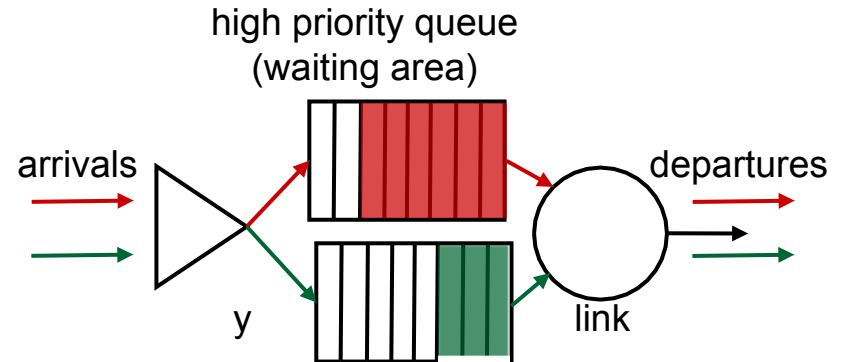


Scheduling policies:

priority

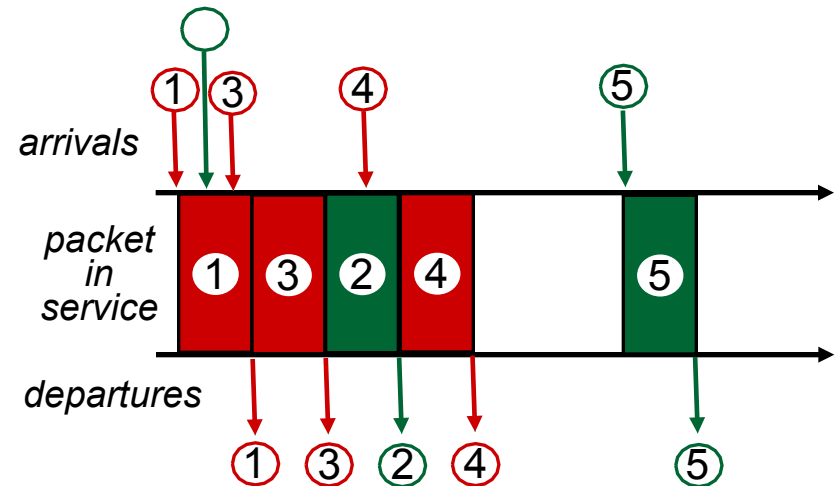
priority scheduling:

send highest
priority queued
packet



prior

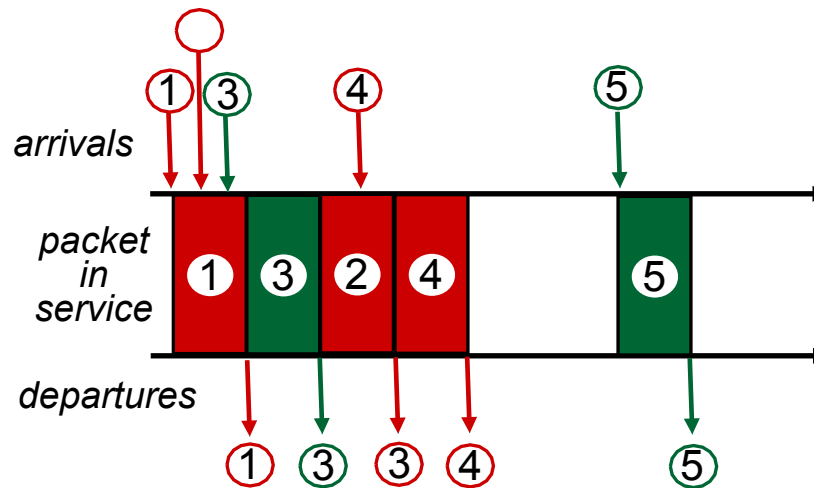
- class may depend on marking or other header info, e.g. IP source/dest, port numbers, etc.
- real world example?



Scheduling policies: still more

Round Robin (RR) scheduling:

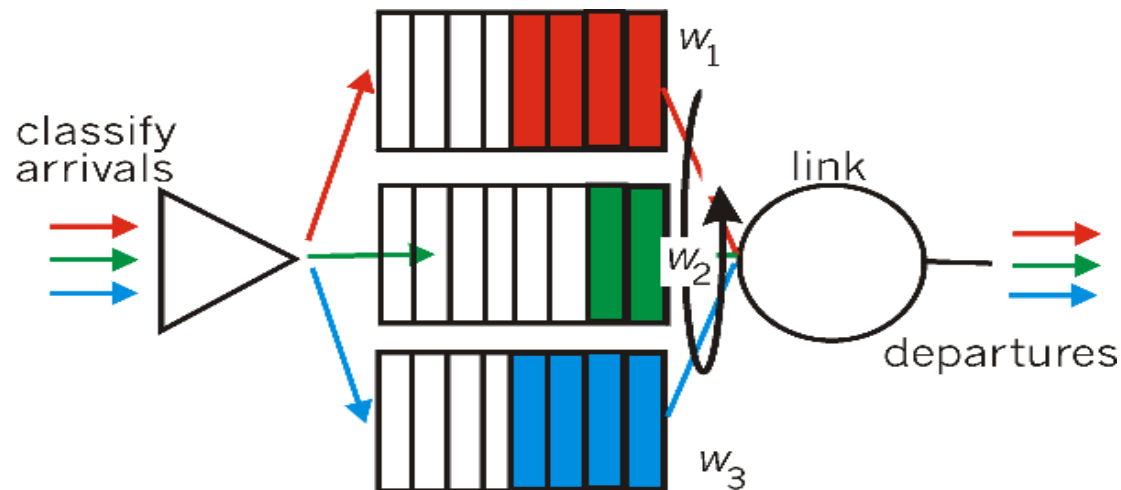
- ❖ multiple classes
- ❖ cyclically scan class queues, sending one complete each class (if



Scheduling policies: still ~~more~~

Weighted Fair Queuing (WFQ):

- ❖ generalized Round Robin
- ❖ each class gets weighted amount of service in ea



Policing mechanisms

goal: limit traffic to not exceed declared parameters

Three common-used criteria:



- *cruc* ngth:
100 packets per sec or 6000 packets per min
have same average!

❖ *peak rate:* e.g., 6000 pkts per min (ppm)
avg.; 1500 ppm peak rate

❖ *(max.) burst size:* max number of pkts
sent consecutively (with no intervening
idle)

Policing mechanisms: ~~implementation~~

token bucket: limit input to specified *burst size* and *average rate*

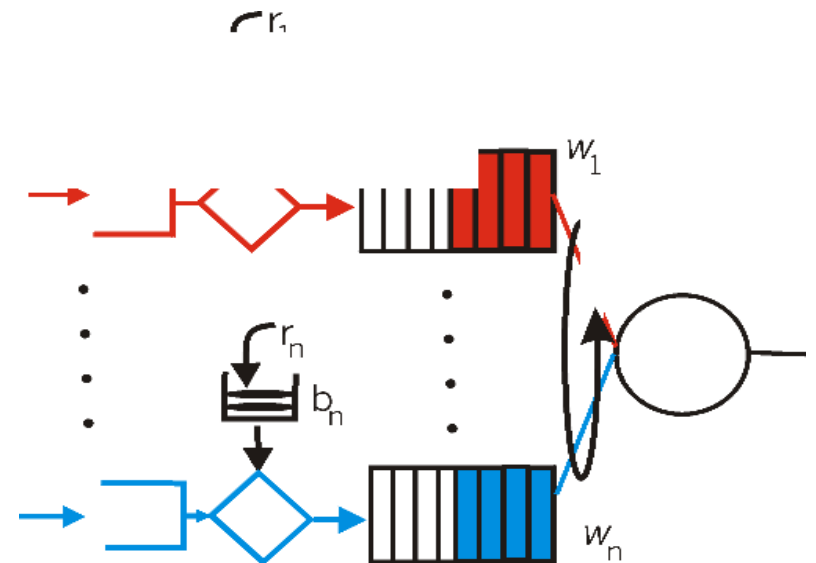
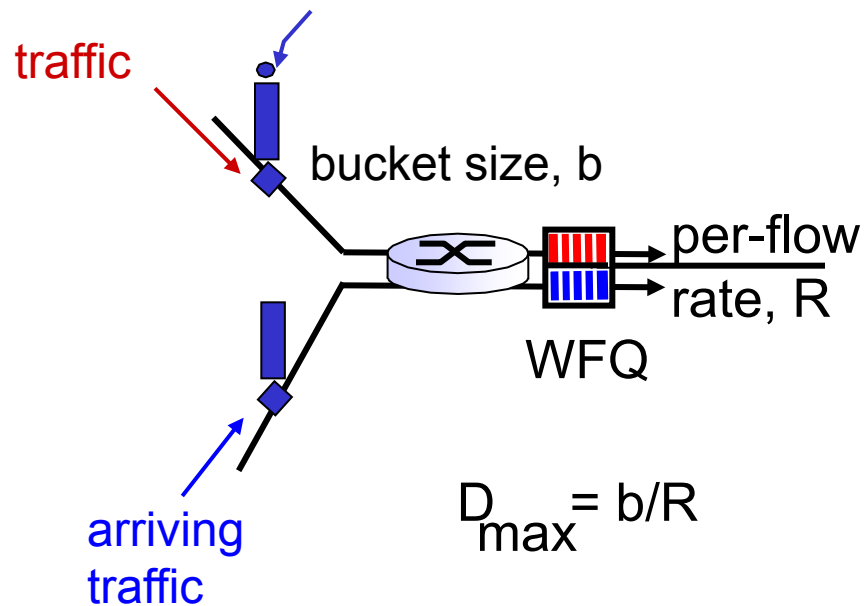


- ❖ bucket can hold b tokens
- ❖ tokens generated at rate r token/sec unless bucket full
- ❖ *over interval of length t : number of packets admitted less than or equal to $(rt + b)$*

Policing and QoS

~~guarantees~~

- ❖ token bucket, WFQ combine to provide guaranteed upper bound on delay, i.e., *QoS guarantee!*



Differentiated services

- ❖ want “qualitative” service classes
 - “behaves like a wire”
 - relative service distinction: Platinum, Gold, Silver

edge

- signaling, maintaining per-flow router state
difficult with large number of flows
- ❖ don't define service classes,
provide functional components to build
service classes

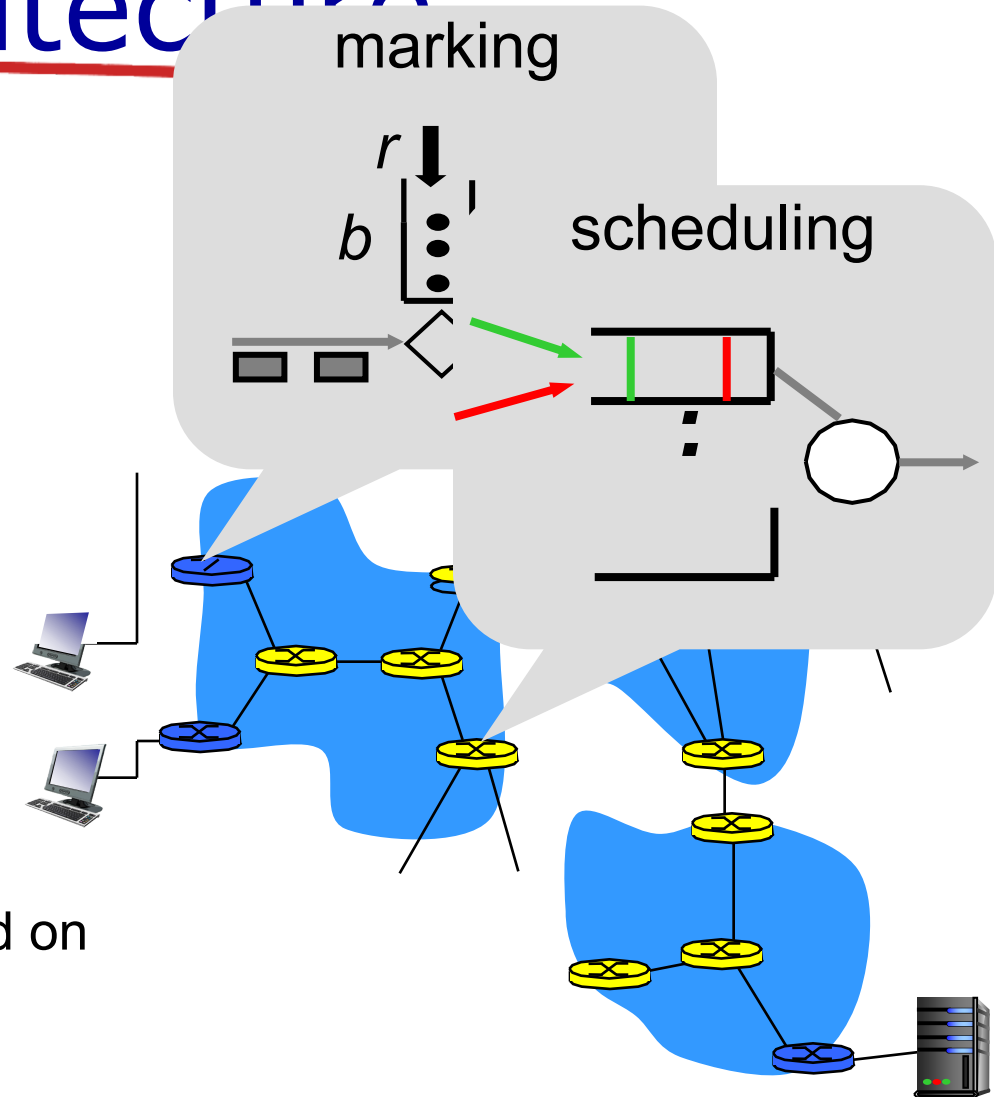
Diffserv architecture

edge router: 

- ❖ *per-flow* traffic management
- ❖ marks packets as **in-profile** and **out-profile**

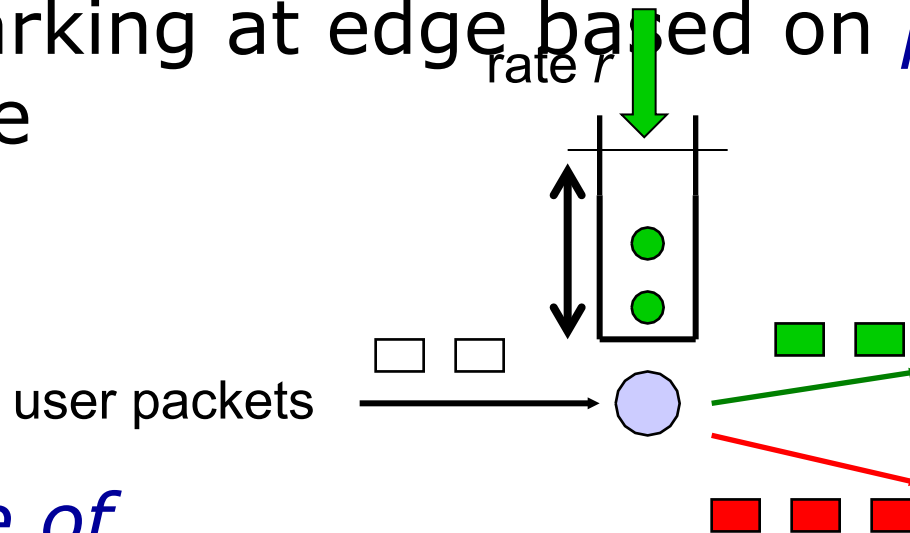
core router: 

- ❖ *per class* traffic management
- ❖ buffering and scheduling based on *marking* at edge
- ❖ preference given to **in-profile** packets over **out-of-profile** packets



Edge-router packet marking

- ❖ *profile*: pre-negotiated rate r , bucket size b
- ❖ packet marking at edge based on *per-flow* profile



possible use of

marking: class-based marking: packets of different classes marked differently

- ❖ intra-class marking: conforming portion of flow marked differently than non-conforming one

Diffserv packet marking: ~~details~~

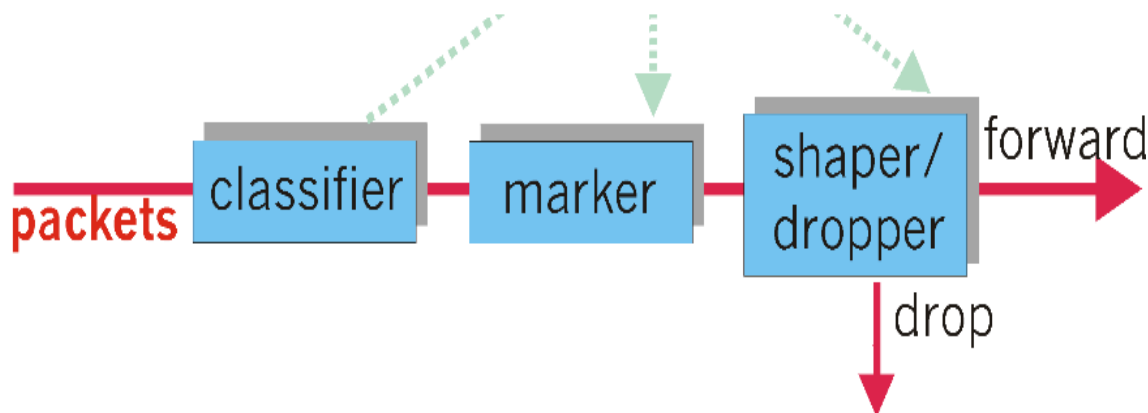
- ❖ packet is marked in the Type of Service (TOS) in IPv4, and Traffic Class in IPv6
- ❖ 6 bits used for Differentiated Service Code Point (
 - 2 b



Classification, conditioning

may be desirable to limit traffic injection rate of some class:

- ❖ user declares traffic profile (e.g., rate, burst size)



Forwarding Per-hop Behavior (PHB)

- ❖ PHB result in a different *observable* (*measurable*) forwarding performance behavior
- ❖ PHB does *not* mechanisms
- ❖ exam
 - class A gets x% of outgoing link bandwidth over time intervals of a specified length
 - class A packets leave first before packets from class B

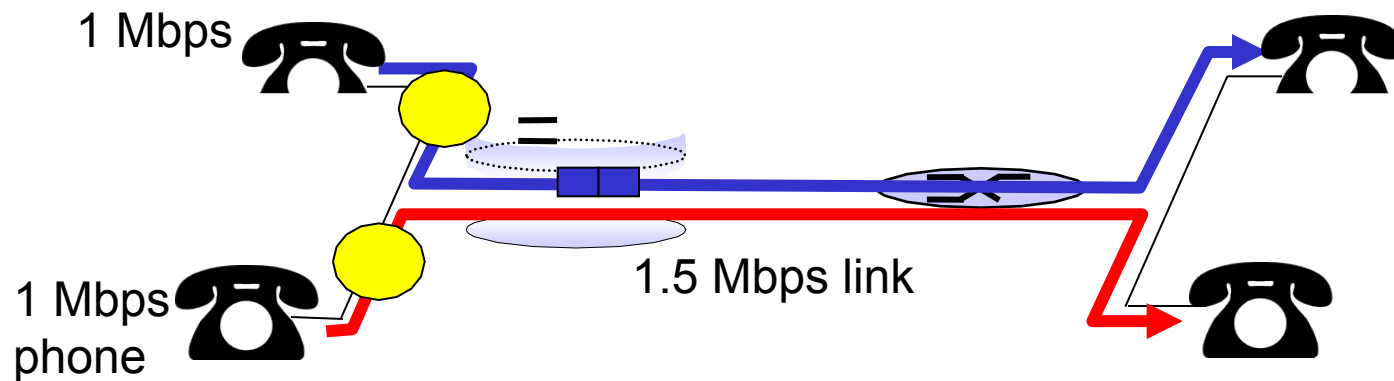
Forwarding PHB

PHBs proposed:

- ❖ *expedited forwarding*: pkt departure
rate of a clas ceeds
- ❖ *assu* ffic
 - each guaranteed minimum amount of bandwidth
 - each with three drop preference partitions

Per-connection QoS guarantees

- ❖ *basic fact of life*: can not support traffic demands beyond link capacity



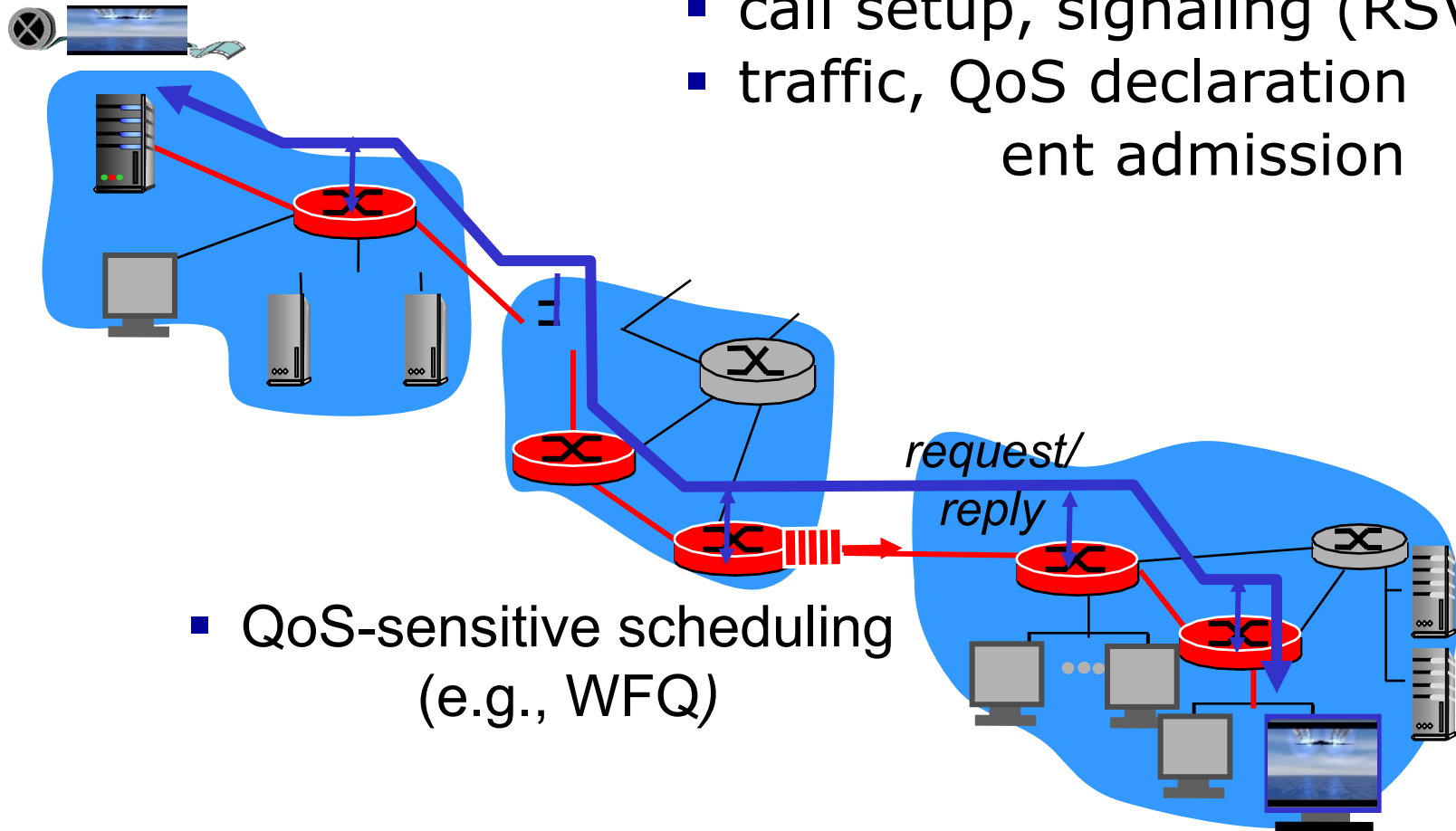
Principle 4

call admission: flow declares its needs, network block call (e.g., busy signal) if it cannot meet

QoS guarantee scenario

❖ *resource reservation*

- call setup, signaling (RSVP)
- traffic, QoS declaration
ent admission



- QoS-sensitive scheduling
(e.g., WFQ)

Multimedia networking: ~~outline~~

7.1 multimedia networking
applications

7.2 streamin 0

7.4 pr
conversational applications

7.5 network support for multimedia